FIAP ON

IOS

PERSISTÊNCIA DE DADOS

6

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 – Adicionando movies.json ao projeto (1) 6
Figura 2 – Adicionando movies.json ao projeto (2) 7
Figura 3 – Adicionando Table View Controller 11
Figura 4 – Adicionando Table View Controller 12
Figura 5 – Definindo classe no Identity Inspector.....13
Figura 6 – Preparando a célula protótipo 15
Figura 7 – Célula customizada.....20
Figura 8 – Assistant Editor em modo Manual.....21
Figura 9 – Persistent Container.....25
Figura 10 – Criando Entity Movie 27
Figura 11 – Atributos de Movie.....27
Figura 12 – Adicionando nova View Controller.....33
Figura 13 – Bar Button Item 34
Figura 14 – Tela de Cadastro de Filme 35
Figura 15 – Tela de Visualização de Filme com botão Edit.....38
Figura 16 – Vinculando a Tab Bar Controller com a nova cena 43
Figura 17 – Storyboard com Tab Bar Controller.....43
Figura 18 – Storyboard com Tab Bar Controller.....44
Figura 19 – Atribuindo a classe à nova cena via Identity Inspector.....45
Figura 20 – Depois do Outlet definindo o Action com Name e Type 46
Figura 21 – Tela de ajustes.....47

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Usando Bundle 7
Código-fonte 2 – Struct Movie 9
Código-fonte 3 – Classe ViewController.....10
Código-fonte 4 – Classe MoviesTableViewController 17
Código-fonte 5 – Métodos do Data Source 18
Código-fonte 6 – Preenchendo células customizadas.....22
Código-fonte 7 – Método prepare.....23
Código-fonte 8 – Classe ViewController.....24
Código-fonte 9 – Extensão de UIViewController com objeto context 28
Código-fonte 10 – Nova implementação de MovieTableViewController.....32
Código-fonte 11 – Nova implementação de ViewController 32
Código-fonte 12 – Classe AddEditViewController 37
Código-fonte 13 – Excluindo filmes 38
Código-fonte 14 – Enviando filme para a tela AddEditViewController.....38
Código-fonte 15 – AddEditViewController 39
Código-fonte 16 – Nova versão de ViewController.....40
Código-fonte 17 – Nova versão de SettingsViewController.....48

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 GRAVANDO INFORMAÇÕES 5
1.1 Bundle 5
1.2 Como trabalhar com JSON 8
1.3 Tabelas: UITableViewController 11
1.3.1 UITableViewController.....11
1.3.2 UITableView 13
1.3.3 UITableViewCell 14
1.3.4 UITableViewDataSource 15
1.3.5 UITableViewDelegate.....16
1.4 Personalisando células.....19
1.4.1 Enviando dados para outra cena.....22
2 PERSISTÊNCIA DE DADOS COM CORE DATA 24
2.1 Core Data Stack 24
2.2 Entity 26
2.3 NSFetchedResultsController 28
2.4 Persistindo dados com UserDefaults 41
REFERÊNCIAS.....49

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 GRAVANDO INFORMAÇÕES

Depois do conteúdo de Auto Layout do capítulo anterior, agora é a vez de deixar o app mais dinâmico. Vamos começar nosso conteúdo falando sobre gravar dados e como recuperá-los, para isso precisamos de um componente muito útil usado para exibir essas informações, estamos falando do componente TableView, avance neste conteúdo e deixe seu app muito mais funcional.

Antes, os aplicativos eram concebidos apenas para entretenimento e muitos eram estáticos, mas, com a evolução das aplicações, hoje temos os aplicativos muito próximos dos sistemas, com operações, conectividade e armazenamento de informações. Neste capítulo, você aprenderá como gravar dados no seu App, localmente. Seja uma informação do seu usuário ou uma lista de dados, além de retornar dados via *JSON*. Preparado para dar continuidade ao seu projeto?

1.1 Bundle

Sempre que adicionamos um arquivo em um projeto iOS, fica armazenado diretamente no *device,* para que possa ser usado dentro do App. Isso é o que ocorre quando adicionamos, por exemplo, imagens em um *Image Set*, porém também é o que ocorre com todo arquivo do projeto, desde os arquivos *Swift* até os *Storyboards*, que ficam em uma área reservada do app chamada **Bundle.**

Um **Bundle** é um diretório com uma estrutura hierárquica padrão que serve para armazenar códigos executáveis e recursos usados por esses códigos. Por fornecer uma localização conhecida e bem estruturada, simplificam a tarefa de acesso a esses recursos.

Quando desejamos inserir arquivos em nosso projeto, utilizamos o objeto **Bundle** para termos um acesso rápido e prático a esse conteúdo.

O **Bundle** principal de todo aplicativo é acessado utilizando a propriedade de classe **main** da classe *Bundle* (**Bundle.main**).

Ao adicionarmos arquivos em nosso *App,* é importante não nos esquecermos de adicioná-lo ao *target* que desejamos utilizar, para que dessa forma, ele seja inserido no **Bundle** do aplicativo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Para continuarmos o desenvolvimento do nosso aplicativo, vamos baixar o arquivo **movies.json** disponibilizado na seção “Asset” da plataforma.

Abra o **projeto MyMovies** (caso tenha fechado) e adicione o arquivo **movies.json** para o projeto. Para isso, arraste pelo **Finder** o arquivo e solte na estrutura de arquivos do seu projeto.

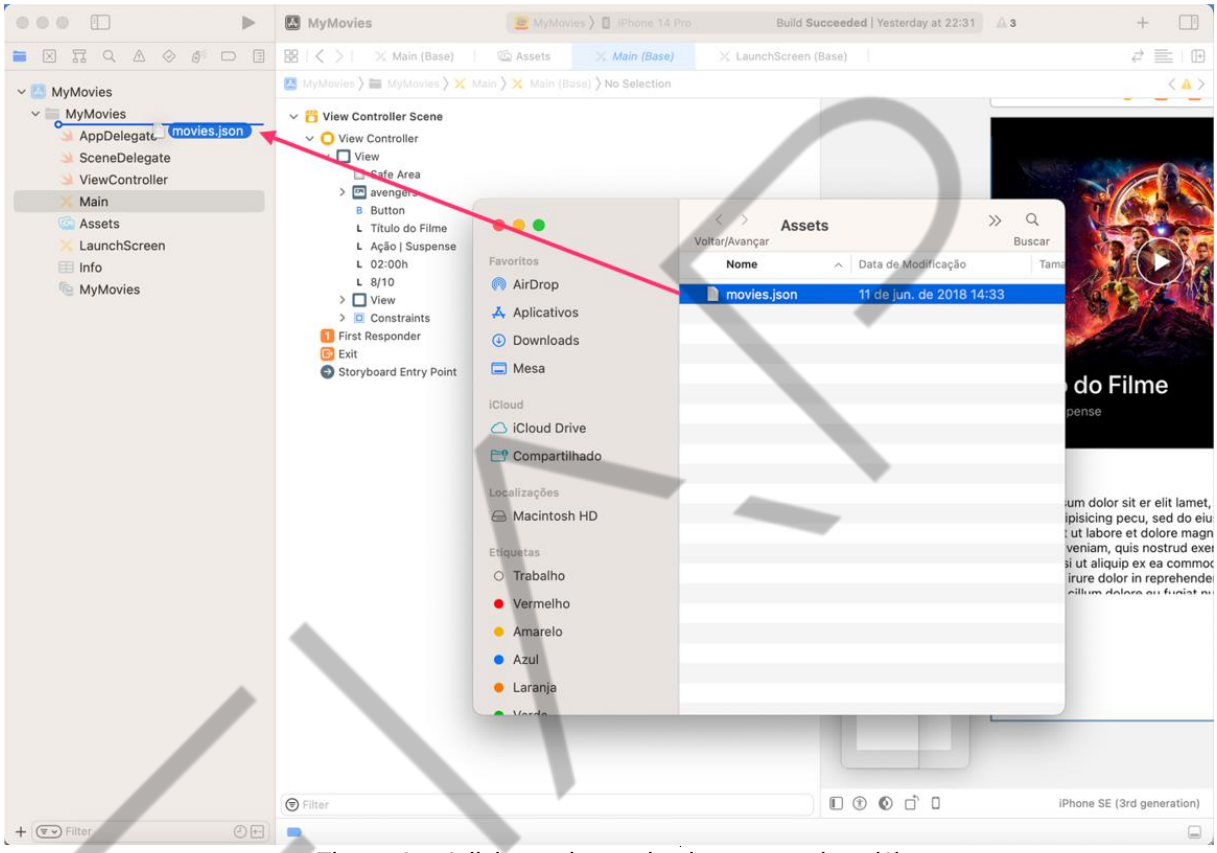

Figura 1 – Adicionando movies.json ao projeto (1)
Fonte: Elaborado pelo autor (2023)

Na janela que se abre, certifique-se de marcar as opções **Copy items if needed** para que o arquivo seja inserido na pasta do projeto, e **Add to targets** para que seja efetivamente inserido no **Bundle**, caso contrário, o arquivo estará no seu projeto, mas não estará disponível no **Bundle** quando o *App* for gerado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

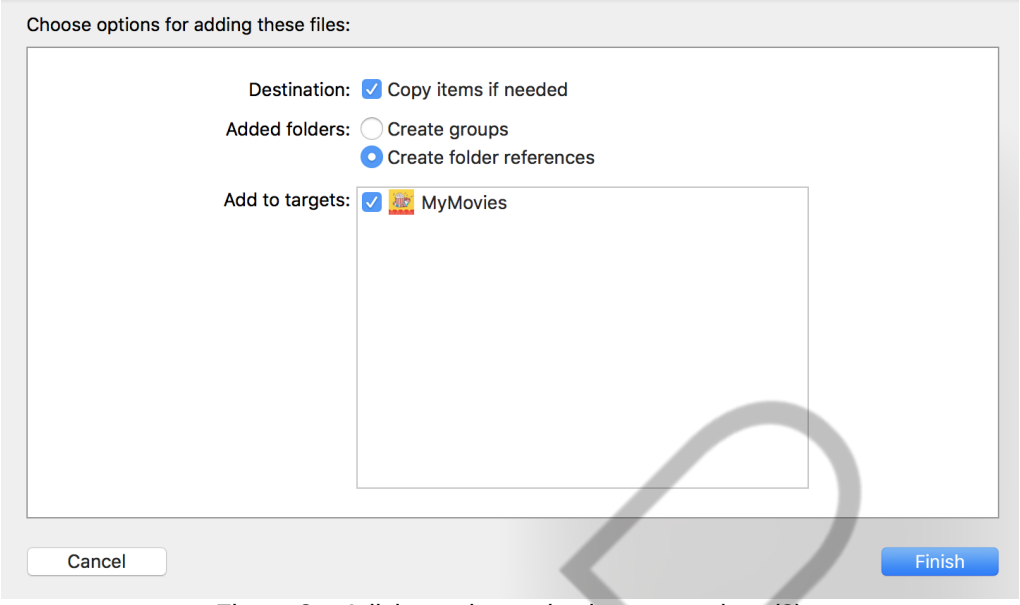

Figura 2 – Adicionando movies.json ao projeto (2)
Fonte: Elaborado pelo autor (2019)

Esse arquivo contém uma lista de filmes e a ideia é recuperar cada um e mostrar ao usuário, fazendo com que possa, posteriormente, visualizar o Filme escolhido, mas, para isso, precisamos ter acesso ao arquivo em nosso código.

Na classe *ViewController,* no seu método *viewDidLoad*, iremos recuperar o arquivo *JSON* para, posteriormente, podermos transformá-lo em uma classe que possamos trabalhar.

Implemente o código-fonte “Usando Budle”.

```
override func viewDidLoad() {
    super.viewDidLoad()
    guard let fileURL = Bundle.main.url(forResource:
            "movies", withExtension: "json") else {return}

    print(fileURL.absoluteString)
}
```

Código-fonte 1 – Usando Bundle
Fonte: Elaborado pelo autor (2019)

Com esse código, estamos recuperando o caminho em que o arquivo se encontra no *device.* Se você rodar o aplicativo agora, verá na *Debug Area* o caminho completo desse arquivo no seu computador (pois estamos rodando no simulador).

Agora que sabemos recuperar o caminho do arquivo, precisamos transformá-lo em algo utilizável, e faremos isso na sequência.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.2 Como trabalhar com JSON

JSON (JavaScript Object Notation) é um modelo para armazenamento e transmissão de informações no formato texto, amplamente utilizado em aplicações web e mobile devido a sua capacidade de estruturar informações de uma forma bem mais compacta do que a conseguida pelo modelo XML.

Apesar de não termos uma classe ou estrutura nativa em *Swift* para o formato *JSON*, podemos converter facilmente estruturas **JSON** em classes *Swift*, fazendo uso do conjunto de protocolos **Codable** e da classe **JSONDecoder**.

Codable é um conjunto de protocolos que deve ser implementado em uma classe para que seja criada a partir de uma estrutura *JSON*, pois contém todas as instruções de como realizar essa operação. É formado pelos protocolos Encodable, utilizado para externar códigos a partir dos seus códigos internos, e o Decodable, utilizado para traduzir códigos externos e trazê-los para dentro do seu código.

A conversão de uma estrutura *JSON* em uma classe *Swift* é feita por meio do método **func decode<T>(_ type: T.Type, from data: Data) throws -> T where T : Decodable**. Esse método faz parte da classe *JSONDecoder*, que foi criada justamente para que fosse fácil recuperar os dados de um *JSON* e transformá-los em uma classe *Swift.*

Nesse método, passamos o nome da classe à qual iremos gerar e o objeto Data que representa o arquivo *JSON* a partir do qual nossa classe será gerada. Para criarmos um objeto Data, que é uma representação em bytes do arquivo, usaremos a classe Data.

Nosso arquivo *movies.json* contém uma lista *(array)* de objetos que representam um filme. Precisamos criar uma classe ou *struct* em *Swift* que represente um filme (*Movie*).

Crie um novo **Swift File** (Command + N) e dê o nome de **Movie**. Dentro dele, implemente o código-fonte “Struct Movie”. Note que, para que possamos posteriormente criar um *Movie* a partir de um *JSON,* além da *struct Movie* ter que implementar o *Codable,* suas propriedades precisam ter exatamente o mesmo nome que as propriedades do filme no *JSON* possuem, ou seja, se nosso objeto console no

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

JSON possui uma propriedade *title* do tipo *String*, nossa *struct Movie* também deverá ter uma propriedade *title* do tipo *String,* e por aí vai.

É importante também notarmos que nosso arquivo é uma representação de um *Array* de filmes, e isso ocorre ao observarmos que o primeiro caractere do arquivo é um colchete ([), o que indica que se trata de uma coleção.

```
import Foundation

struct Movie: Codable {
    var title: String
    var categories: String
    var duration: String
    var rating: Double
    var summary: String
    var image: String
}
```

Código-fonte 2 – Struct Movie
Fonte: Elaborado pelo autor (2019)

Agora, vamos transformar nosso *JSON* em um *Array de Movie.* Primeiro, precisamos criar uma variável (**movies: [Movie]= []**) que armazenará esse *array.* Após isso, vamos criar um método para fazer essa leitura de conversão e chamá-lo no *viewDidLoad* de nossa *ViewController.*

Esse método recuperará a *URL* do nosso arquivo e irá transformá-lo em bytes, um *Data,* para que possamos manipulá-lo. De posse desse *Data*, instanciamos a classe *JSONDecoder* e usamos o seu método *decode,* informando que aquele *Data* será decodificado em um **[Movie].self**, ou seja, um *Array de Movie*, afinal de contas, é isso que nosso *JSON* é, um *array de Movie.* Precisamos colocar .self para a conversão funcionar.

É importante notar que tanto o método *decode* quanto o construtor de *Data* disparam exceção, portanto, devem ser utilizados dentro de um bloco **Do Catch**. No final, sua classe *ViewController* ficará como apresentado no código-fonte “Classe ViewController”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
import UIKit

class ViewController: UIViewController {

    var movies: [Movie] = []

    override func viewDidLoad() {
        super.viewDidLoad()

        //Chamamos o método loadMovies()
        loadMovies()
    }

    //Este método abrirá o arquivo movies.json e converterá
    //  em um array de Movie
    func loadMovies() {

        //Recuperando a URL do arquivo
        guard let fileURL = Bundle.main.url(forResource:
              "movies", withExtension: "json") else {return}
        do {

            //Criando o dara, a representação binária de
            // nosso arquivo
            let data = try Data(contentsOf: fileURL)

            //Decodificando o JSON em um Array de Movie
            movies = try JSONDecoder().decode([Movie].self,
                     from: data)

            //Varremos o array para imprimir na Debug Area o
            //  nome e duração de cada filme
            for movie in movies {
                print(movie.title, "-", movie.duration)
            }
        } catch {
            //Caso aconteça algum erro na interpretação do
            // JSON, o sistema de tratamento exibirá a causa
            print(error.localizedDescription)
        }
    }
}
```

Código-fonte 3 – Classe ViewController
Fonte: Elaborado pelo autor (2019)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.3 Tabelas: UITableViewController

1.3.1 UITableViewController

Um dos componentes mais utilizados no desenvolvimento de aplicações iOS é o *Table View,* que é a representação de listas em iOS. Com ele, podemos apresentar listas de informações, possibilitando sua seleção e até mesmo exclusão.

Existe uma *controller* desenvolvida especialmente para trabalhar com *Table Views,* a **UITableViewController**. Esta *controller* implementa os protocolos necessários para que possamos alimentar e configurar um *Table View*, que são os protocolos **UITableViewDataSource** e **UITableViewDelegate**.

Ao criamos uma **UITableViewController**, já temos configurada uma *Table View* para utilizar. Vamos adicionar ao nosso projeto uma *Table View Controller*. Para isso, filtre o *Library por Table View Controller* e arraste para o *Storyboard.* Posicione essa *Table View Controller* do lado esquerdo da nossa cena atual.

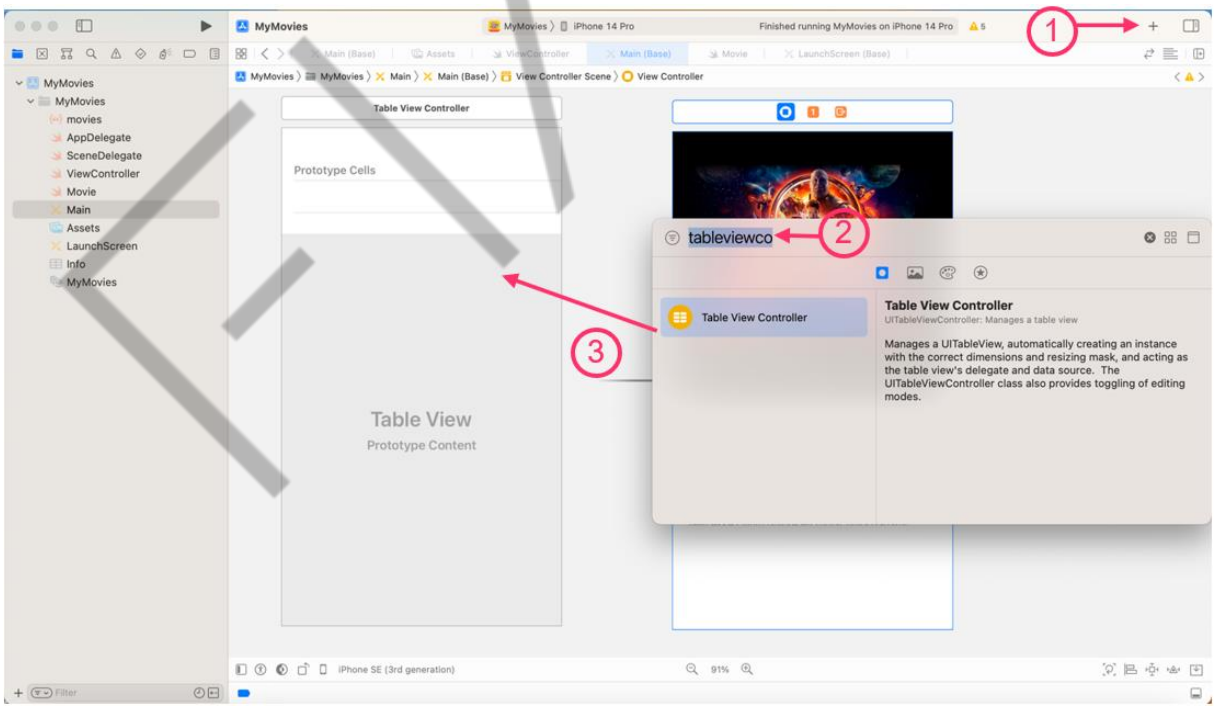

Figura 3 – Adicionando Table View Controller
Fonte: Elaborado pelo autor (2019)

Coloque essa *Table View Controller* dentro de uma *Navigation Controller (*selecione o DragonBall da Table View Controller e clique no Menu em Editor →

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Embed In – Navigation Controller) e tire o *Storyboard Entry* Point da cena atual e passe para a *Navigaton Controller.* Dessa forma, será a tela inicial do nosso App agora.

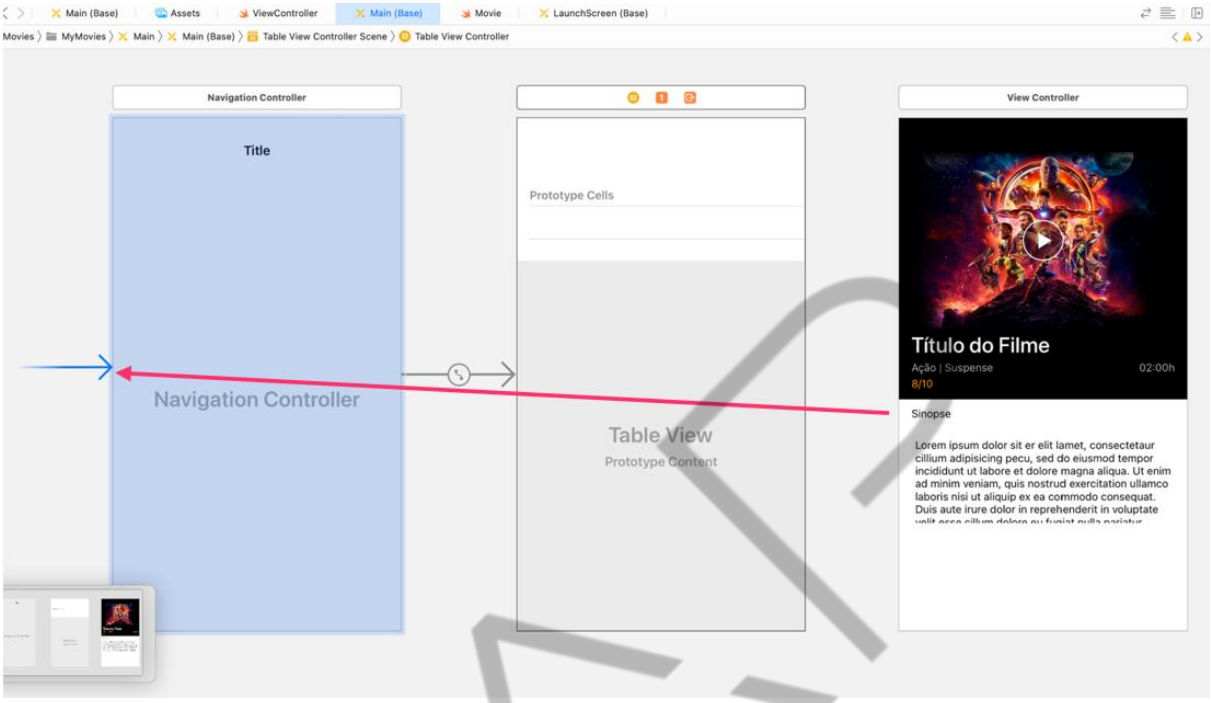

Figura 4 – Adicionando Table View Controller
Fonte: Elaborado pelo autor (2023)

Crie uma nova **Cocoa Touch Class** (Command + N) que herdará **UITableViewController** e dê o nome de **MoviesTableViewController**.

Defina que a *UITableViewController* adicionada no *Storyboard* terá como classe a recém-criada *MoviesTableViewController* (selecione a *DragonBall* da *Table View Controller*, vá ao *Identity Inspector* e coloque *MoviesTableViewController* como sendo sua classe).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

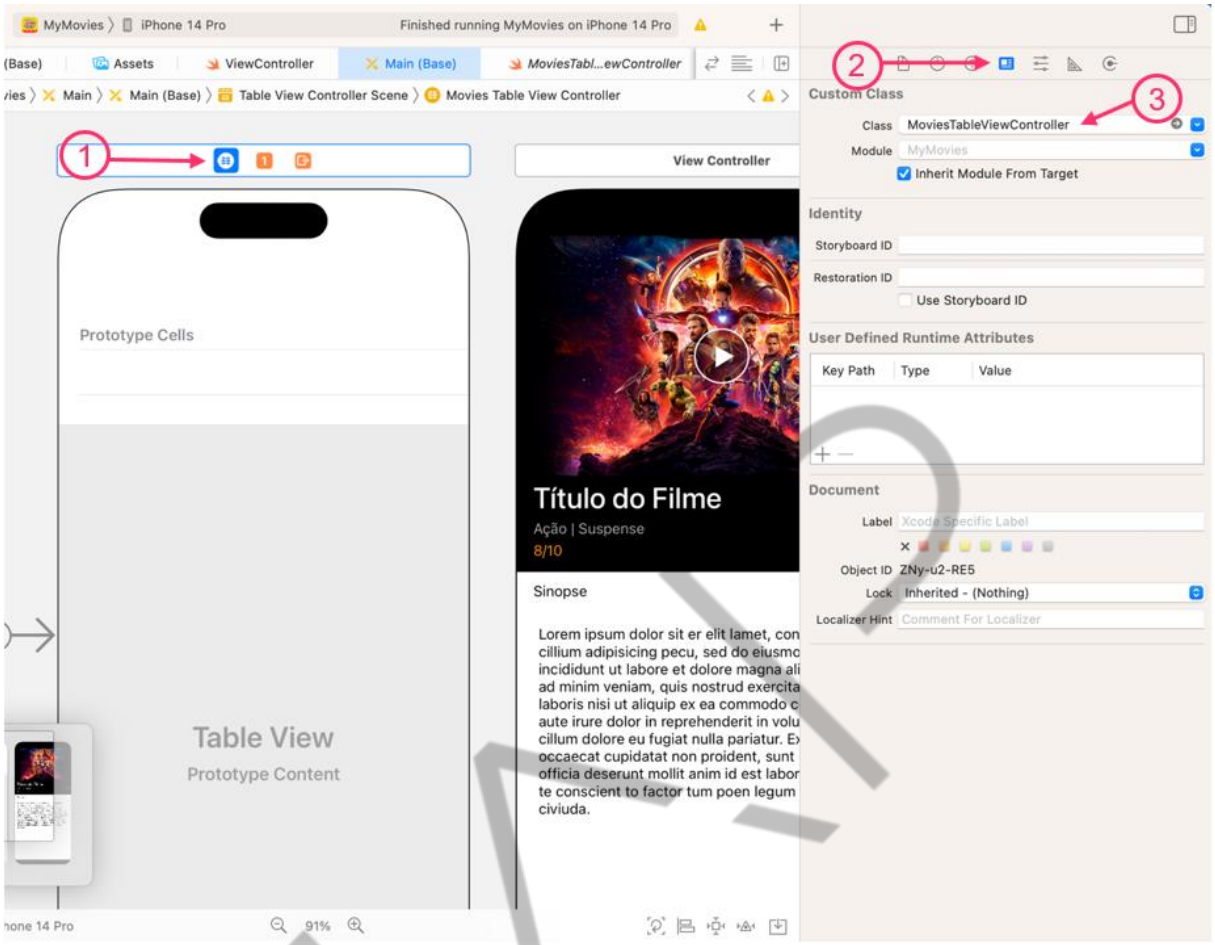

Figura 5 – Definindo classe no Identity Inspector
Fonte: Elaborado pelo autor (2023)

1.3.2 UITableView

No coração da *Table View Controller* encontra-se um dos principais componentes presentes no desenvolvimento de aplicações iOS, a *Table View.*

Uma **UITableView** é um objeto utilizado quando precisamos mostrar uma lista hierárquica de itens, com possibilidade de edição, seleção, reordenação etc. *Table Views* apresentam seus elementos apenas em uma coluna, com possibilidade de *scroll* na vertical.

Cada item de uma *Table View* é representado por uma *Table View Cell* (**UITableViewCell**). Essas células podem ser agrupadas em seções para fácil identificação.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Uma *Table View* precisa ter um objeto que aja como *delegate* (responsável por executar ações de configurações e reagir a eventos do usuário) e também um objeto que aja como *DataSource* (para configurar a fonte de dados que alimentará a tabela).

O objeto que servirá como *DataSource* deve implementar o protocolo **UITableViewDataSource**, e quem agirá como *delegate* deve implementar o protocolo **UITableViewDelegate**. Ao utilizarmos uma *UITableViewController* (lembre-se de que criamos uma classe chamada *MoviesTableViewController* que herdou *UITableviewController*), ambos protocolos são implementados por ela, ou seja, não precisamos nos preocupar em implementá-los.

1.3.3 UITableViewCell

Cada uma das células apresentadas por uma *Table View* é representada pela classe **UITableViewCell**. Esta classe possui propriedades para definir e gerenciar o conteúdo e aparência dessas células.

Podemos criar células customizadas ou utilizar estilos prontos, que fornecem *labels* e imagens com estilos e posições predefinidos, que podem ser acessados pelas propriedades **textLabel** (para o texto principal), **detailTextLabel** (informação secundária) e **imageView** (imagem associada à célula).

Dentre os estilos, temos:

- **Basic**: textLabel alinhado à esquerda.
- **Right Detail**: textLabel à esquerda e detailTextLabel à direita.
- **Left Detail**: textLabel à esquerda e detailTextLabel ao lado.
- **Subtitle**: textLabel à esquerda e detailTextLabel abaixo.

Para trabalharmos com células, precisamos definir seu **Identifier**, que será um rótulo associado àquela célula protótipo e será usado na sua criação ou reutilização. Uma *Table View* pode ter várias células protótipo, com estilos diferentes. Cada uma deve possuir um *Identifier* próprio e único. Na grande maioria das vezes, usamos apenas uma célula protótipo em uma *Table View.*

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Selecione a célula protótipo da *Table View* e defina seu estilo como **Right Detail**, e seu *Identifier* como **cell**. Isso é muito importante para que possamos recuperar essa célula no nosso código.

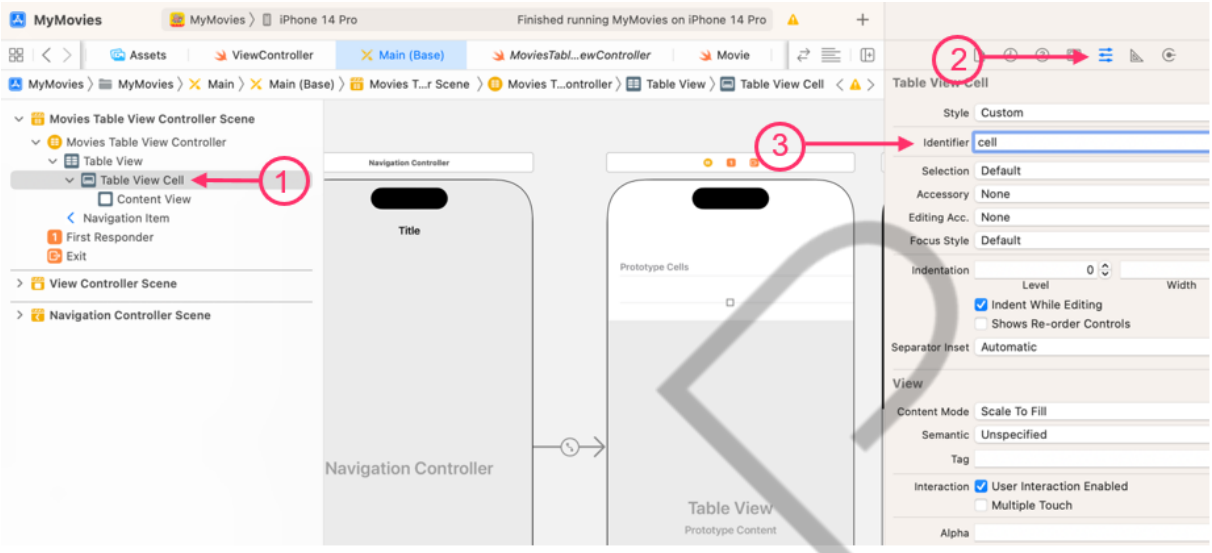

Figura 6 – Preparando a célula protótipo
Fonte: Elaborado pelo autor (2023)

1.3.4 UITableViewDataSource

O protocolo **UITableViewDataSource** é implementado automaticamente pela classe *UITableViewController* e é utilizado para os objetos que farão o papel de mediar o modelo de dados para uma tabela *(Table View).*

O *DataSource* fornece a uma *Table View* as informações necessárias para a sua construção e modificação.

Dentre os métodos que esse protocolo fornece, estão aqueles para definir a quantidade das seções que uma tabela possui, quantas células cada seção irá conter, bem como as informações que cada uma dessas células apresentará.

A maneira como os métodos de *DataSource* nos fornecem informações de qual célula estamos trabalhando é por meio de um objeto do tipo **IndexPath**. Esse objeto possui propriedades que contém informações da seção à qual aquela célula pertence (**.section**) bem como em qual linha (**.row**) se encontra.

Ao utilizarmos uma *UITableViewController,* os principais métodos de *UITableViewDataSource* estão implementados e/ou comentados no código.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Os métodos que obrigatoriamente utilizaremos para nossa tabela são:

- **func numberOfSections(in tableView: UITableView) -> Int**: Solicita quantas seções a tabela irá conter.
- **func tableView(_ tableView: UITableView, numberOfRowsInSection section: Int) -> Int**: Solicita quantas linhas (células) cada seção da tabela apresentará.
- **func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) -> UITableViewCell**: Solicita a célula que será apresentada em determinado indexPath (objeto que contém as informações de seção e linha de determinada célula).

Ao criarmos a célula que será retornada no método cellForRowAt indexPath, usamos o método **func dequeueReusableCell(withIdentifier identifier: String, for indexPath: IndexPath) -> UITableViewCell**, que retorna uma célula reutilizável, aumentando assim a performance do nosso App, pois cria somente o número necessário de células para a tabela.

1.3.5 UITableViewDelegate

Outro protocolo que a **UITableViewController** implementa é o **UITableViewDelegate**, que define que tal classe é *delegate* da *Table View.* O *delegate* de uma *Table View* é responsável, dentre outras coisas, por gerenciar seleções de células, configurar os cabeçalhos e rodapés das seções e auxiliar na exclusão e reordenação das células.

Alguns exemplos dos métodos são:

- **func tableView(UITableView, heightForRowAt: IndexPath)**: Define a altura para determinada célula.
- **func tableView(UITableView, didSelectRowAt: IndexPath)**: Informa que a célula foi selecionada.
- **func tableView(UITableView, viewForHeaderInSection: Int)**: Solicita uma *view* que será utilizada como cabeçalho da seção especificada.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **func tableView(UITableView, editActionsForRowAt: IndexPath)**: Solicita quais ações serão apresentadas quando for feito o *Swipe para a esquerda* na célula.

Agora, vamos preparar nossa *Table View* para mostrar os filmes presentes no *JSON.* A primeira coisa que temos que fazer é retirar o código que implementamos na *ViewController* e repassar para a classe *MoviesTableViewController,* ou seja, recorte a definição do *array movies,* o método *loadMovies* e a chamada desse método na *viewDidLoad* e cole na classe *MoviesTableViewController.* Também retire o que usamos para imprimir os filmes. Nesse momento, o início da classe ficará como mostra o código-fonte "Classe MoviesTableViewController":

```
import UIKit

class MoviesTableViewController: UITableViewController {

    var movies: [Movie] = []

    override func viewDidLoad() {
        super.viewDidLoad()
        loadMovies()
    }

    func loadMovies() {
        guard let fileURL = Bundle.main.url(forResource:
                "movies", withExtension: "json") else {return}
        do {
            let data = try Data(contentsOf: fileURL)
            movies = try JSONDecoder().decode([Movie].self,
                        from: data)
        } catch {
            print(error.localizedDescription)
        }
    }
```

Código-fonte 4 – Classe MoviesTableViewController
Fonte: Elaborado pelo autor (2019)

Agora vamos preparar nossa *Table View* implementando os métodos do protocolo *UITableViewDataSource.* Implemente os métodos conforme o código-fonte "Métodos do Data Source" (não se esqueça de descomentar o método **tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) -> UITableViewCell**

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

que está comentado logo abaixo de **tableView(_ tableView: UITableView, numberOfRowsInSection section: Int) -> Int**).

```
// MARK: - Table view data source

    override func numberOfSections(in tableView: UITableView)
             -> Int {
        //Número de seções da nossa tabela. Marcamos como 1.
        return 1
    }

    override func tableView(_ tableView: UITableView,
             numberOfRowsInSection section: Int) -> Int {
        //Número de linhas da tabela, que é o número de
        //filmes que nosso array contém
        return movies.count
    }

    //Este método prepara cada uma das células para
    //apresentar o filme correspondente
    override func tableView(_ tableView: UITableView,
      cellForRowAt indexPath: IndexPath) -> UITableViewCell {


        //Usamos o mesmo identifier definido no storyboard
        let cell =
tableView.dequeueReusableCell(withIdentifier: "cell", for:
indexPath)

        //Usamos a propriedade row do IndexPath para
        //recuperar o índice da célula, que é o mesmo índice
        //do movie no array
        let movie = movies[indexPath.row]

        //Configuramos o nome e a duração
        cell.textLabel?.text = movie.title
        cell.detailTextLabel?.text = movie.duration

        //Retornamos a célula que será apresentada
        return cell
    }
```

Código-fonte 5 – Métodos do Data Source
Fonte: Elaborado pelo autor (2023)

Note como preparamos a célula baseada no identificador **cell**, recuperamos o **movie** relativo ao seu *IndexPath* e depois atribuímos o nome e duração nos campos **textLabel** e **detailTextLabel**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.4 Personalisando células

Nós utilizamos células padrão do sistema, porém é possível usar uma célula personalizada com os elementos que quiser. Para isso, primeiro selecione a célula e, no *Attributes Inspector*, defina o *Style* como **Custom**. Depois, vá ao *Size Inspector* e defina o *Row Height* como **118**. Agora, vamos personalizar nossa célula da seguinte maneira:

- Mude a cor de fundo para preto.
- Adicione uma *Image View* e defina seu frame como X: **16**, Y: **11**, Width: **70** e *Height:* **89**.
- Crie suas *constraints* de **topo**, **esquerda**, **largura** e **altura**, deixando os valores como estão.
- Adicione uma *Label* **branca**, com texto Nome Filme, tamanho **16** e fonte **System Semibold**, defina seu frame como X: **98**, Y: **11**, *Width:* **210** e *Height:* **20**. O número de linhas *(Lines)* deverá ser definido como **0** e crie as *constraints* de **topo**, **esquerda** e **direita**.
- Adicione uma *Label* de cor laranja, tamanho **14***, frame* como X: **98**, Y: **37**, *Width*: **210** e *Height:* **17**. O número de linhas (*Lines)* deverá ser definido como **0** e crie as *constraints* de **topo**, **esquerda**, **direita** e **base** deixando os valores como estão.
- Agora, selecione a *constraint* de **base** dessa *label* (clique na linha azul abaixo dela) e no *Size Inspector* mude o *Relation* para **Greater Than or Equal**, logo abaixo, troque o valor de *Constant* para **8**. Com isso, estamos dizendo que essa *constraint* deve ter um tamanho maior ou igual a **8**, pois, caso o filme tenha uma sinopse curta, a *label* ficará no topo, e se a sinopse for maior crescerá e empurrará a célula para baixo, aumentando o tamanho dela, mas mantendo uma distância de **8** dessa *label.*
- Para finalizar, selecione a *Image View* e adicione uma *constraint* de base a ela. Isso fará com que, se a sinopse for curta demais, a célula não reduza e corte a imagem, porém se deixarmos o tamanho fixo, a célula nunca irá crescer, portanto, após adicionar essa *constraint,* deverá selecioná-la

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

(aumente o zoom para poder selecionar essa constraint, ou selecione pelo *Document Outline*) e mudar também o seu *Relation* para **Greater Than or Equal**.

Agora, sua tela deverá estar como mostra a Figura “Célula customizada”.

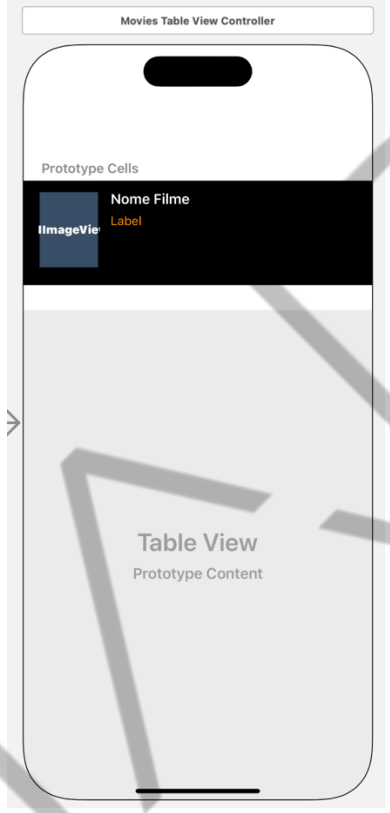

Figura 7 – Célula customizada
Fonte: Elaborado pelo autor (2023)

Para que possamos utilizar essa célula, temos que criar uma classe do tipo **UITableViewCell** e atribuir à ela. Crie uma nova **Cocoa Touch Class (Command + N)**, Subclass of **UITableViewCell**, com nome de **MovieTableViewCell**.

Selecione a célula e atribua essa classe à ela pelo Identity Inspector. Agora, temos que criar os *outlets* para podermos manipular esses elementos. Abra o *Assistant Editor.* Notará que a classe que aparece é a *MoviesTableViewController,* porém, precisamos que seja a *MovieTableViewCell*. Para isso, na *Navigator Area (Project Navigator)* clique, com o botão *Option* pressionado, no arquivo **MovieTableViewCell.** Isso fará com que seja aberto do lado direito do Assistant Editor, nos permitindo criar nossos outlets a partir da célula. Quando fazemos isso, o Assistant Editor entra em modo Manual.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

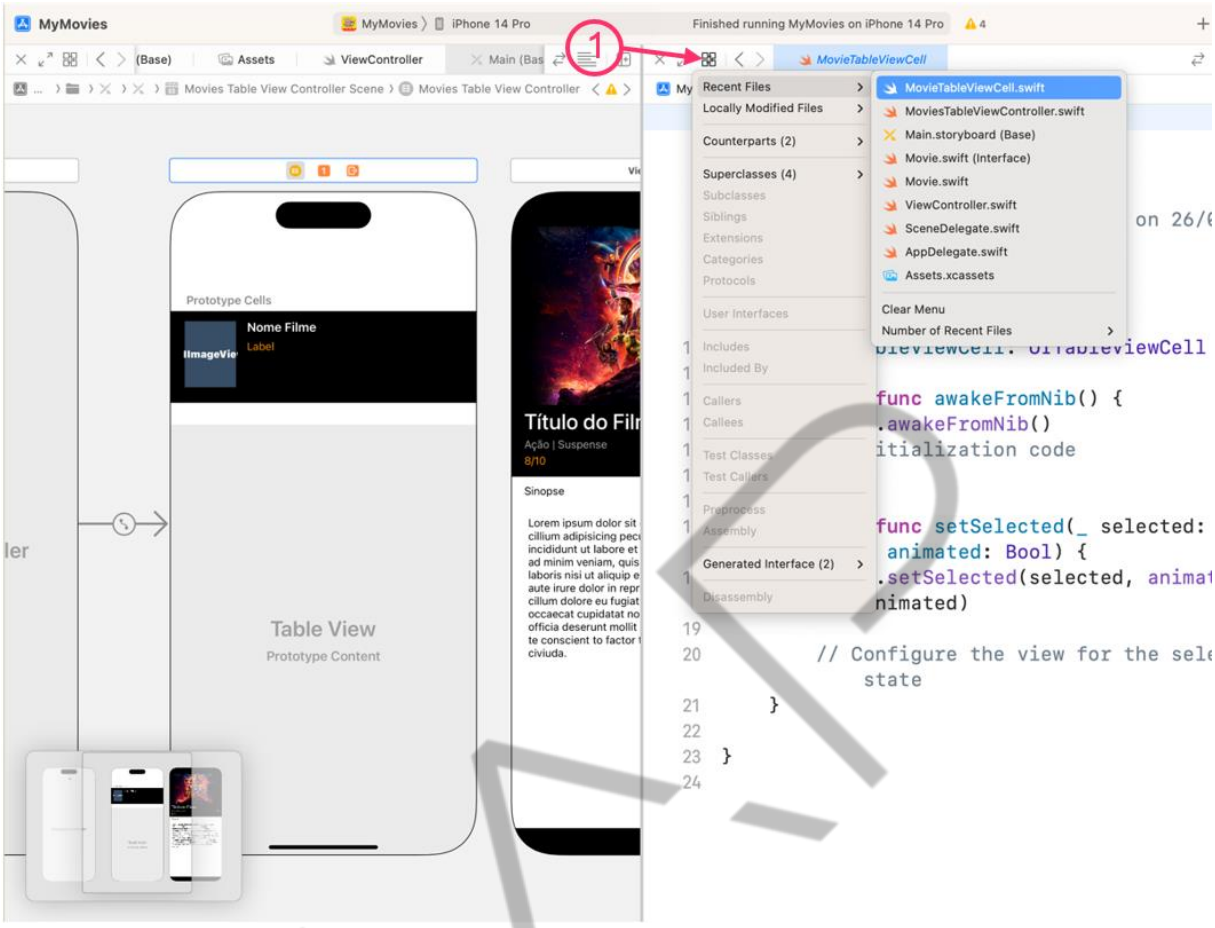

Figura 8 – Assistant Editor em modo Manual
Fonte: Elaborado pelo autor (2023)

Crie três *outlets,* um para cada elemento da célula. O da *Image View* será **ivMovie**, do *Label* de título será **lbTitle** e do *Label* de sinopse será **lbSummary**. Feito isso, mude o *Assistant Editor* de Manual para **Automatic** apenas fechando a janela de código do **MovieTableViewCell**.

Agora, precisamos alterar a forma como mostramos as informações na tabela. Não podemos mais usar *textLabel* e *detailTextLabel* pois, agora, nossa célula é customizada. Primeiro, precisamos tratar nossa célula como sendo uma *MovieTableViewCell* para, assim, termos acesso aos nosso *outlets.* Modifique o método *cellForRowAt* para que fique como no código-fonte “Preenchendo células customizadas”. Leia as explicações presentes no próprio código.

```
override func tableView(_ tableView: UITableView,
cellForRowAt indexPath: IndexPath) -> UITableViewCell {

        //Vamos tratar a célula como sendo uma
        //MovieTableViewCell
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        let cell =
tableView.dequeueReusableCell(withIdentifier: "cell", for:
indexPath) as!
              MovieTableViewCell

        let movie = movies[indexPath.row]

        //Agora podemos modificar os outlets da célula.

        //Para definirmos uma imagem via código, modificamos
        // a propriedade image da Image View.
        //Esta propriedade define qual imagem será usada. As
        //imagens são do tipo UIImage e criamos uma passando
        //o nome da imagem no Assets.
        cell.ivMovie.image = UIImage(named:
                                "\(movie.image)")
        cell.lbTitle.text = movie.title
        cell.lbSummary.text = movie.summary

        return cell
    }
```

Código-fonte 6 – Preenchendo células customizadas
Fonte: Elaborado pelo autor (2023)

Agora, nosso App mostra a lista dos filmes com suas respectivas imagens!

1.4.1 Enviando dados para outra cena

A ideia é que o usuário possa tocar em um filme e visualizar suas informações de maneira detalhada, para isso, temos que repassar os dados de um filme para uma nova tela.

Isso é feito utilizando um método chamado **prepare (for segue: UIStoryboardSegue, sender: Any?)**, que faz parte de toda *UIViewController* e tem a função de preparar uma cena antes de aparecer. Nesse método, podemos recuperar o filme selecionado e repassar seu objeto para a próxima tela, preenchendo os dados.

Esse método nos dá acesso à cena e, a partir dela, podemos recuperar a próxima tela por meio da propriedade **destination**. Implemente o código-fonte "Método prepare" na classe **MoviesTableViewController,** logo abaixo de **viewDidLoad.**

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
override func prepare(for segue: UIStoryboardSegue, sender:
Any?) {
    //Recuperando a próxima cena e tratando-a como
    //ViewController

    if let vc = segue.destination as? ViewController {

        //A propriedade indexPathForSelectedRow nos retorna o
        //IndexPath da célula selecionada. Usaremos como
        //índice para indicar o movie selecionado
        let movie =
               movies[tableView.indexPathForSelectedRow!.row]

        //Repassando o movie para a próxima tela
        vc.movie = movie
    }
}
```

Código-fonte 7 – Método prepare
Fonte: Elaborado pelo autor (2019)

Você deve ter percebido um erro, pois a classe *ViewController* não possui a propriedade *movie.* Vamos adicionar essa propriedade à classe *ViewController* e aproveitar para preencher os dados dessa tela, caso tenha recebido um filme, mas, para isso, deve criar os *outlets* para cada um dos elementos da cena (imagem, *labels* de título, gêneros, duração e nota, e *text view).* Para cada componente da tela, crie os *outlets* conforme os nomes que se encontram no código-fonte "Classe ViewController". Esse código é a versão final de como sua classe *ViewController* deverá ficar (comente ou retire o código *loadMovies()* que estava no viewDidLoad()).

```
import UIKit

class ViewController: UIViewController {

    var movie: Movie?

    @IBOutlet weak var ivMovie: UIImageView!
    @IBOutlet weak var lbTitle: UILabel!
    @IBOutlet weak var lbCategories: UILabel!
    @IBOutlet weak var lbDuration: UILabel!
    @IBOutlet weak var lbRating: UILabel!
    @IBOutlet weak var tvSummary: UITextView!

    override func viewDidLoad() {
        super.viewDidLoad()
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        if let movie = movie {
            ivMovie.image = UIImage(named: movie.image)
            lbTitle.text = movie.title
            lbCategories.text = movie.categories
            lbDuration.text = movie.duration
            lbRating.text = "☆ \(movie.rating)/10"
            tvSummary.text = movie.summary
        }
    }
}
```

Código-fonte 8 – Classe ViewController
Fonte: Elaborado pelo autor (2019)

Agora, para testar, crie uma segue da célula até a segunda tela, escolhendo **show**. Rode o App e perceba que, ao clicar em um filme, você visualizará todas as suas informações na tela seguinte.

2 PERSISTÊNCIA DE DADOS COM CORE DATA

Existem diversas formas de persistirmos dados em uma aplicação iOS, mas a mais robusta é usando banco de dados.

Core Data é um *framework* utilizado para trabalhar a camada **Model** de sua aplicação. Ele nos permite persistir dados em nosso aplicativo, utilizando banco de dados, de uma forma orientada a objetos.

Core Data não é um banco de dados, mas sim um gerenciador de grafos que também inclui persistência. Trabalha (agrupa, filtra e organiza) os dados em memória.

2.1 Core Data Stack

Chamamos de *Core Data Stack* o conjunto de ferramentas e classes que utilizamos para trabalhar com o *Core Data.* Essas classes dividem-se em:

- **Managed Object Model**: Descreve o *schema* que usa no seu App. Podemos imaginá-lo como sendo o *schema* de um banco de dados, mas

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

sendo representado por uma coleção de objetos (também conhecidos como entidades). É definido por um arquivo com a extensão .xcdatamodeld.

- **Persistent Store Coordinator**: Responsável pelo gerenciamento da *Persistent Store,* que nos dá acesso ao banco de dados *(SQLite por padrão)* que persistirá nossos objetos.
- **Managed Object Context**: É uma área de trabalho em memória que utilizamos para manipular os objetos (criar, alterar, salvar, recuperar etc.) que serão persistidos na *Persistent Store* (banco de dados).

Todos esses objetos são gerenciados por um objeto chamado **Persistent Container**.

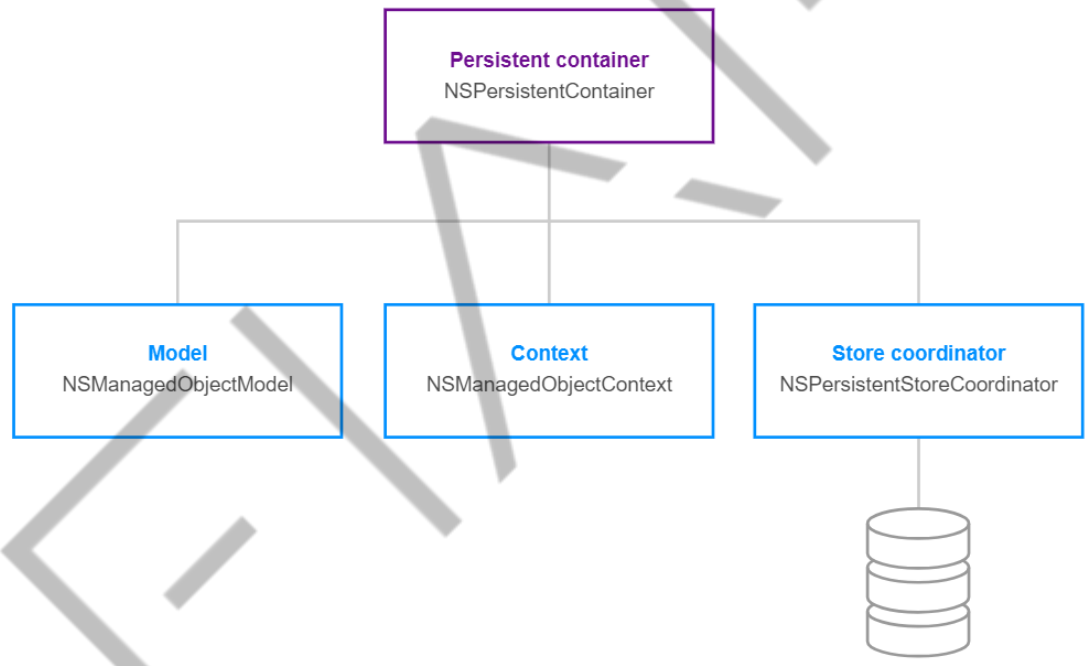

Figura 9 – Persistent Container
Fonte: Documentação Core Data Apple (2020)

Quando criamos nosso projeto e marcamos para usar *Core Data*, o *Xcode* criou nosso *Data Model*, que é o arquivo *MyMovies.xcdatamodeld* (atualmente essa extensão ".xcdatamodeld" não aparece, aparecerá apenas se você clicar em MyMovies e apertar Enter). Além disso, criou no *AppDelegate* nosso *Persistent Container*, então, o que precisamos fazer é abrir nosso *Data Model* e criar nossa *Entity* que representará um *Movie* no nosso App. A partir de agora, nosso App não irá mais mostrar os filmes a partir do *JSON*, mas nós iremos cadastrar nossos próprios filmes

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

no *Core Data* e listá-los. Assim, nossa lista pode ter quantos filmes desejarmos e não se limitará mais aos do *JSON.*

Antes de começarmos, precisamos excluir nossa *struct Movie.* Apague o arquivo **Movie.swift** e ignore, por enquanto, os erros que irão aparecer no App (selecione o arquivo, aperte DELETE e escolha **Move to Trash**).

Selecione o arquivo **MyMovies.xcdatamodeld** e siga os próximos passos para criar a entidade**.**

2.2 Entity

Para que possamos começar a persistir filmes no *Core Data*, precisamos criar uma representação de um *Movie* no *Core Data*, e isso é feito por meio de *Entities*. Uma **Entity** em *Core Data* representa uma classe. Em um banco de dados relacional, uma *Entity* representa uma tabela.

Uma **Entity** possui atributos que representam as propriedades da classe. Fazendo uma relação com banco de dados, um atributo representa um campo em uma tabela.

Crie uma entidade chamada **Movie**. Para isso, clique no botão **Add Entity** e renomeie a entidade criada.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

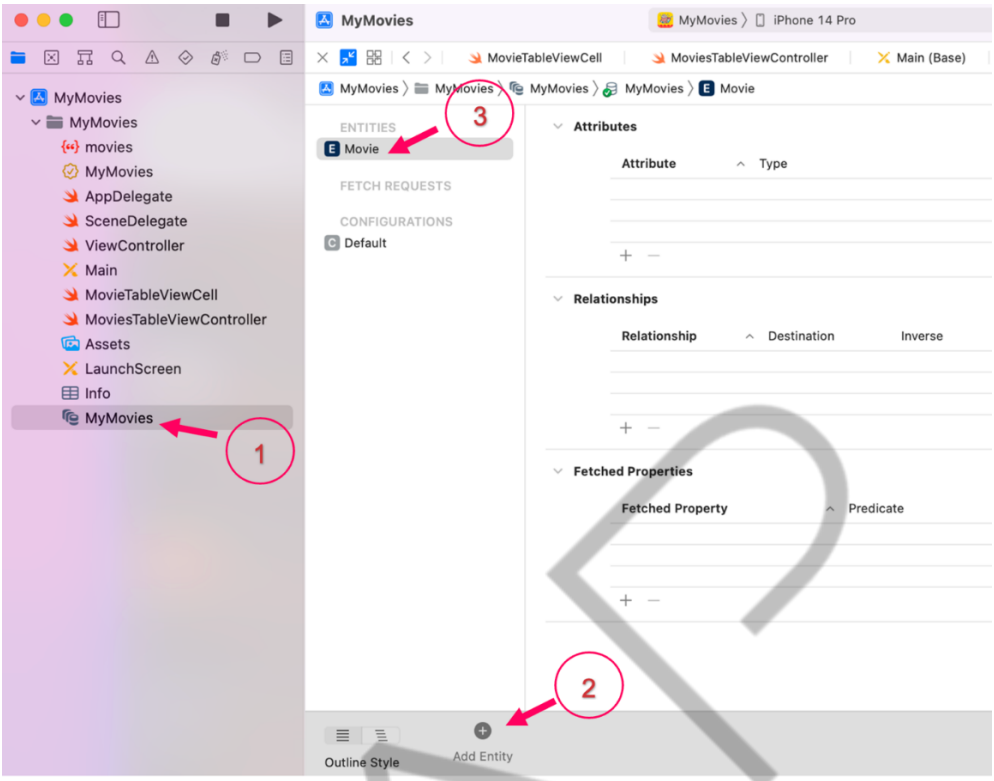

Figura 10 – Criando Entity Movie
Fonte: Elaborado pelo autor (2019)

Precisamos criar os atributos das nossas **Entities**. Um atributo de uma *Entity* corresponde às propriedades de uma classe. Para adicionar um atributo, clique no botão **Add Items** e altere o nome (**Attribute**) e tipo (**Type**) do atributo criado.

Vamos criar os seguintes atributos (todos com letras minúsculas) para a **Entity Movie:** *duration: String, image: Transformable, rating: Double, summary: String, title: String e categories: String.*

Obs.: O tipo *Transformable* é muito útil quando vamos usar imagens. Iremos armazenar o *poster* do filme nesse atributo.

No final, a *entity Movie* ficará como mostra a Figura “Atributos de Movie”.

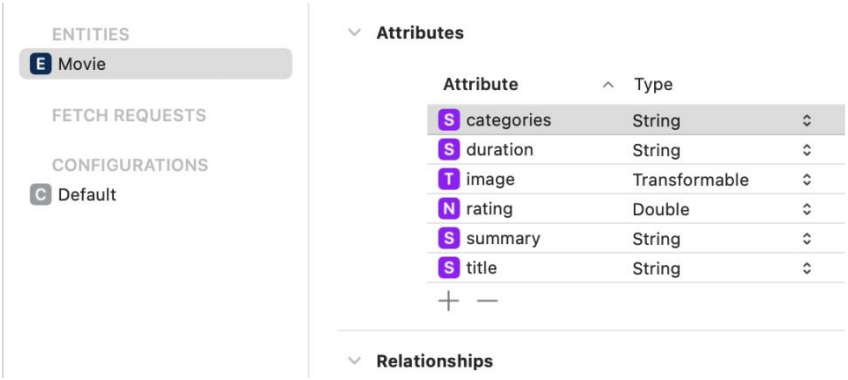

Figura 11 – Atributos de Movie
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Ao longo do uso do *Core Data,* precisaremos ter acesso ao *Managed Object Context,* que é a área na qual todas as informações do *Core Data* são manipuladas. Temos acesso a esse objeto por meio do objeto *persistentContainer* que foi criado na classe *AppDelegate.* Para facilitar, vamos criar uma extensão para as classes *UIViewController* e, nessa extensão, vamos criar uma variável chamada *context*, que nos dará acesso direto ao *Managed Object Context.* **Crie um *Swift File*** (Command + N) e chame-o de *UIViewController+CoreData*. Dentro dele, implemente o código-fonte "Extensão de UIViewController com objeto contexto".

```
import UIKit
import CoreData

extension UIViewController {

    //Esta variável computada nos dará acesso ao
    //NSManagedObjectContext a partir de qualquer tela
    var context: NSManagedObjectContext {

        //Aqui estamos criando uma referência ao AppDelegate
        let appDelegate = UIApplication.shared.delegate
             as! AppDelegate



        //Conseguimos acessar o NSManagedObjectContext a
        //partir da propriedade .viewContext presente em
        //nosso persistentContainer. Aqui apenas retornamos
        //essa propriedade.
        return appDelegate.persistentContainer.viewContext
    }
}
```

Código-fonte 9 – Extensão de UIViewController com objeto context
Fonte: Elaborado pelo autor (2019)

2.3 NSFetchedResultsController

Vamos, agora, implementar a leitura dos filmes cadastrados. Apesar de não termos nenhum filme cadastrado no banco, vamos deixar a leitura pronta.

Para isso, precisamos de um objeto do tipo **NSFetchedResultsController**. Ele será responsável por solicitar, ao contexto, a lista de filmes cadastrados. Esse objeto

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

possui um *delegate* que é informado quando qualquer alteração de filmes for feita no contexto (edição, exclusão, adição).

Deixe sua classe *MoviesTableViewController* conforme o código-fonte “Nova implementação de MovieTableViewController”. Atente para as explicações presentes no código e para o ***import CoreData*** no topo. Precisamos importar esse *framework* para termos acesso às suas funcionalidades.

```
import UIKit
import CoreData

class MoviesTableViewController: UITableViewController {

    //Criamos o objeto fetchedResultsController definindo
    //que será um controller da entidade Movie
    var fetchedResultsController:
                        NSFetchedResultsController<Movie>!

    override func viewDidLoad() {
        super.viewDidLoad()
        loadMovies()
    }

    override func prepare(for segue: UIStoryboardSegue,
      sender: Any?) {
        if let vc = segue.destination as? ViewController {
            let movie = fetchedResultsController.object(at:
                        tableView.indexPathForSelectedRow!)
            vc.movie = movie
        }
    }

    func loadMovies() {

        //O objeto fetchRequest é responsável por fazer uma
        //leitura dos itens do Core Data. Criamos um
        //fetchRequest de Movie pois queremos buscar todos os
        //filmes da base. A classe Movie (gerada pelo Core
        //Data) já possui um método que nos retorna seu
        //fetchRequest
        let fetchRequest: NSFetchRequest<Movie> =
Movie.fetchRequest()

        //Abaixo, definimos que os filmes serão ordenados
        //em ordem alfabética pelo título
        let sortDescriptor = NSSortDescriptor(key: "title",
ascending: true)
        fetchRequest.sortDescriptors = [sortDescriptor]
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        //Instanciando objeto fetchedResultsController. Aqui
        //precisamos passar o fetchRequest e o contexto do
        //Core Data.
        fetchedResultsController =
           NSFetchedResultsController(fetchRequest:
           fetchRequest, managedObjectContext: context,
           sectionNameKeyPath: nil, cacheName: nil)

        //Definimos que esta classe será delegate do
        //fetchedResultsController. Ela que será chamada
        //quando algo acontecer no contexto dos filmes
        fetchedResultsController.delegate = self
        do {
            //Executando a requisição de movies
            try fetchedResultsController.performFetch()
        } catch {
            print(error.localizedDescription)
        }
    }

    // MARK: - Table view data source

    override func numberOfSections(in tableView: UITableView)
        -> Int {
        return 1
    }

    override func tableView(_ tableView: UITableView,
                numberOfRowsInSection section: Int) -> Int {
        //Os filmes estarão presentes no objeto
        //fetchedObjects. Verificamos se tem filmes
        //e caso não tenha retornamos 0
        return fetchedResultsController.fetchedObjects?.count
           ?? 0
    }

    //Este método prepara cada uma das células para
    //apresentar o filme correspondente
    override func tableView(_ tableView: UITableView,
      cellForRowAt indexPath: IndexPath) -> UITableViewCell {

        //Vamos tratar a célula como sendo uma
        //MovieTableViewCell


    let cell = tableView.dequeueReusableCell(
            withIdentifier: "cell", for: indexPath) as!
                    MovieTableViewCell

        //Para recuperarmos o filme, usamos o método
        //object(at: ) do fetchedResultsController
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        //Com ele, passamos o indexPath da célula e obtemos o
        //filme vinculado a ela
        let movie = fetchedResultsController.object(at:
                    indexPath)

        //Para preencher a imagem, basta tratarmos o atributo
        //image como sendo uma Imagem,
        //pois nele nós salvaremos a imagem do filme
        cell.ivMovie.image = movie.image as? UIImage
        cell.lbTitle.text = movie.title
        cell.lbSummary.text = movie.summary

        return cell
    }

    /*
    // Override to support editing the table view.
    override func tableView(_ tableView: UITableView, commit
          editingStyle: UITableViewCell.EditingStyle,
forRowAt
              indexPath: IndexPath) {
        if editingStyle == .delete {
            // Delete the row from the data source
            tableView.deleteRows(at: [indexPath], with:
             .fade)
        } else if editingStyle == .insert {
            // Create a new instance of the appropriate
              class, insert it into the array, and add a new
              row to the table view
        }
    }
    */
}

//Através de extensions, vamos implementar o protocolo
//NSFetchedResultsControllerDelegate para que esta classe
//possa ser delegate do
objeto:fetchedResultsControllerDelegate
extension MoviesTableViewController:
                        NSFetchedResultsControllerDelegate {

    //O método abaixo é chamado sempre que alguma alteração é
    //feita em algum filme. Quando isto ocorrer, iremos
    //fazer o reload da tabela, assim ela recarrega todos os
    //filmes e atualiza seu conteúdo.
    func controllerDidChangeContent(_ controller:
          NSFetchedResultsController<NSFetchRequestResult>) {
        tableView.reloadData()
    }
}
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Código-fonte 10 – Nova implementação de MovieTableViewController
Fonte: Elaborado pelo autor (2019)

Há mais um erro que precisamos corrigir, que é na classe *ViewController*, pois lá ainda estamos tratando a imagem como sendo uma *String.* Ela deverá ficar como mostra o código-fonte “Nova implementação de ViewController”.

```
import UIKit

class ViewController: UIViewController {

    var movie: Movie?

    @IBOutlet weak var ivMovie: UIImageView!
    @IBOutlet weak var lbTitle: UILabel!
    @IBOutlet weak var lbCategories: UILabel!
    @IBOutlet weak var lbDuration: UILabel!
    @IBOutlet weak var lbRating: UILabel!
    @IBOutlet weak var tvSummary: UITextView!

    override func viewDidLoad() {
        super.viewDidLoad()

        if let movie = movie {
            ivMovie.image = movie.image as? UIImage
            lbTitle.text = movie.title
            lbCategories.text = movie.categories
            lbDuration.text = movie.duration
            lbRating.text = "☆ \(movie.rating)/10"
            tvSummary.text = movie.summary
        }
    }
}
```

Código-fonte 11 – Nova implementação de ViewController
Fonte: Elaborado pelo autor (2019)

Vamos agora criar a tela de cadastro de filmes. Adicione uma nova *View Controller* ao projeto e posicione essa *controller* em cima da *Table View Controller,* como mostra a Figura “Adicionando nova View Controller”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

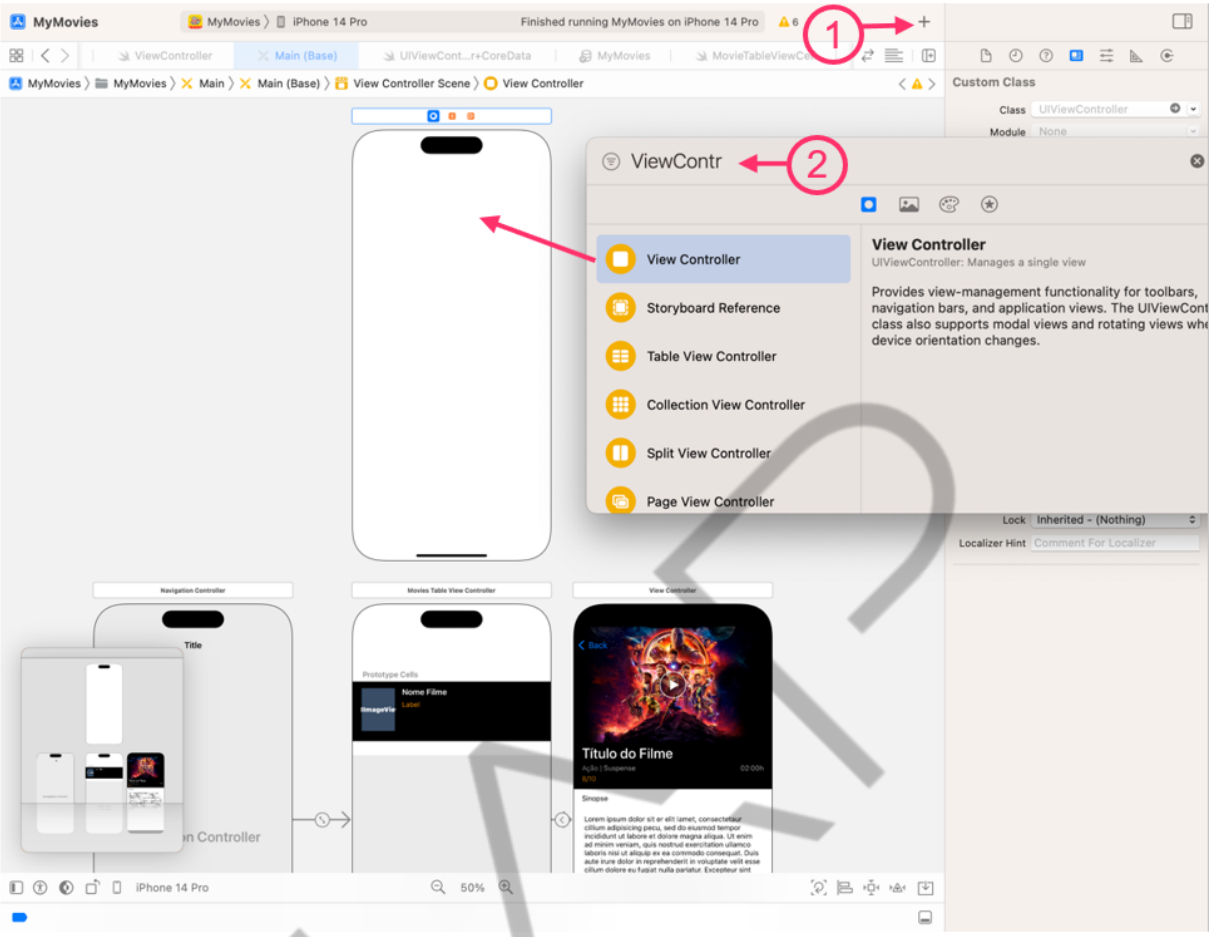

Figura 12 – Adicionando nova View Controller
Fonte: Elaborado pelo autor (2023)

Em *Library* digite *Bar Button* Item, adicione um *Bar Button* Item no *navigation bar* da nossa *Table View Controller* do lado direito. *Bar Button* Item é o botão que utilizamos dentro de *Navigation Bars.*

Depois de adicionado, altere sua propriedade System Item para **Add**, para que ele se pareça com um sinal de **+**. Agora, crie uma segue (**show**) desse botão até a nova *View Controller.*

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

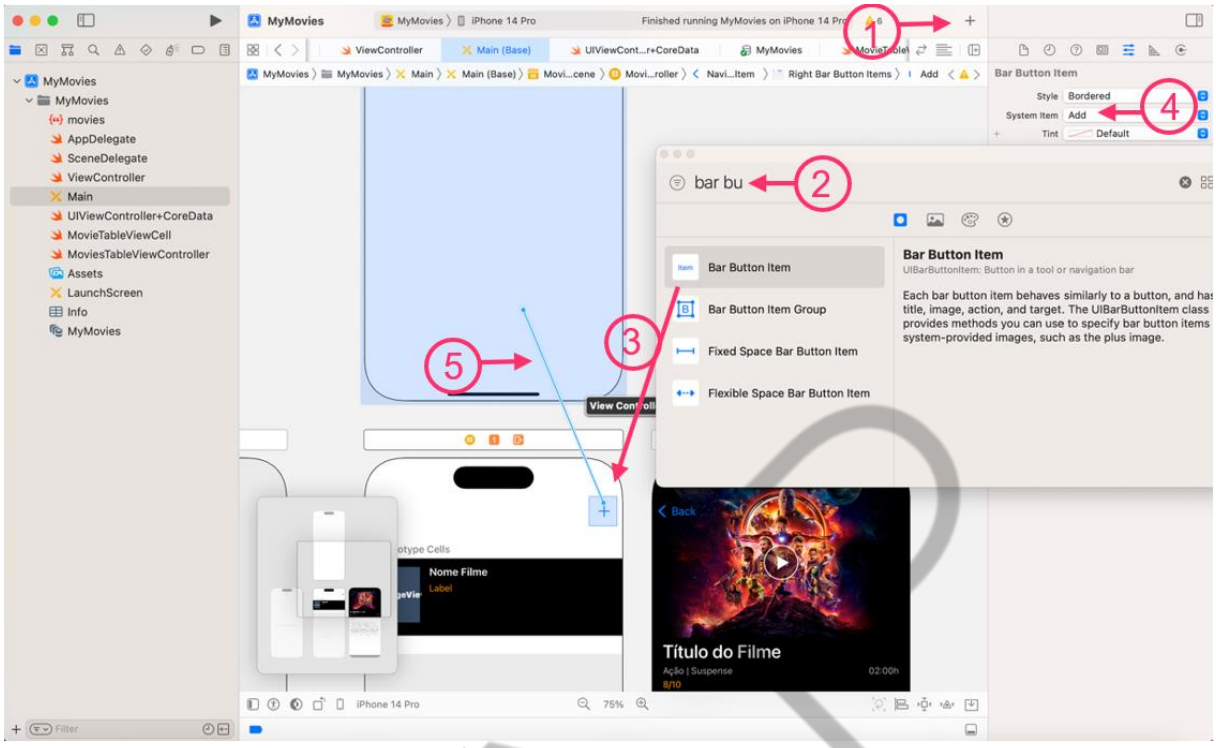

Figura 13 – Bar Button Item
Fonte: Elaborado pelo autor (2023)

Preencha a sua nova *View Controller* da seguinte forma:

- Adicione um *Text Field* com o *placeholder* **Título** e *frame* X: **16**, Y: **120**, *Width:* **361**. Crie as *constraints* de topo, esquerda e direita.
- Altere a cor de fundo da View para preto.
- Adicione um *Text Field* com o *placeholder* **Duração** e *frame* X: **297**, Y: **170**, *Width*: **80**. Crie as *constraints* de topo, direita e *Width.*
- Adicione um *Text Field* com o *placeholder* **Nota** e *frame* X: **229**, Y: **170**, *Width:* **46**. Crie as *constraints* de topo, direita e *Width.*
- Adicione um *Text Field* com o placeholder **Categorias** e *frame* X: **16**, Y: **170**, *Width:* **194**. Crie as *constraints* de topo, esquerda e direita.
- Adicione um *Label* **branco** com o texto **Sinopse**, tamanho **15** e *frame* X: **16**, Y: **219**, *Width*: **55** e *Height*: **18**. Crie as *constraints* de topo, esquerda e *Width.*
- Adicione um Text View com frame X: **89**, Y: **219**, Width: **288** e Height: **90**. Crie as constraints de topo, esquerda e direita e Height, limpe o conteúdo do Text View.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Adicione um *Label* **branco** com o texto **Imagem**, tamanho **15** e *frame* X: **16**, Y: **321**, *Width:* **55** e *Height:* **18**. Crie as *constraints* de topo, esquerda e *Width.*
- Adicione um *Image View* com *frame* X: **89**, Y: **315**, *Width*: **285** e *Height*: **137**. Crie as *constraints* de topo, esquerda e direita e *Height*.
- Adicione um *Button* com texto **Cadastrar** cor **branco**, fundo laranja e com *frame* X: **16**, Y: **473**, *Width*: **358** e *Height:* **35**. Crie as *constraints* de topo, esquerda e direita e *Height.*
- Por último, adicione um *Button* com texto **Adicionar Pôster** de cor **branco**, fundo azul e *frame* X: **89**, Y: **315**, *Width*: **285** e *Height*: **137**. Crie as *constraints* de topo, esquerda e direita e *Height.*

Sua tela ficará como mostra a Figura “Tela de Cadastro de Filme”.

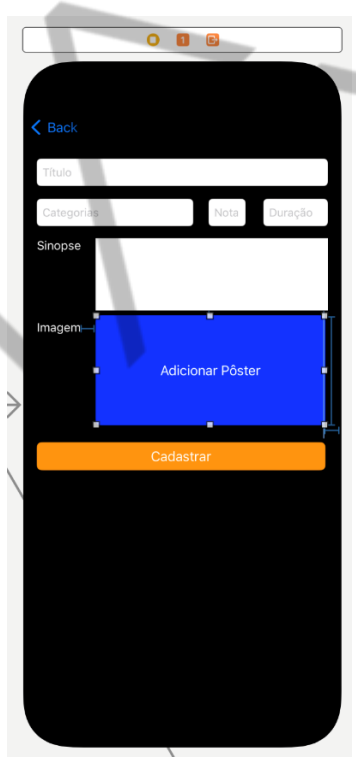

Figura 14 – Tela de Cadastro de Filme
Fonte: Elaborado pelo autor (2023)

Agora, crie a *Cocoa Touch Class (Commnad + N)* com o nome de **AddEditViewController**, herdando *UIViewController* e vincule a essa nova cena pelo *Identity Inspector*. Crie os *outlets, actions* e implemente o código-fonte “Classe AddEditViewController”.

```
import UIKit
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
class AddEditViewController: UIViewController {

    //Variável que receberá o filme selecionado.
    var movie: Movie?

    @IBOutlet weak var ivMovie: UIImageView!
    @IBOutlet weak var tfTitle: UITextField!
    @IBOutlet weak var tfCategories: UITextField!
    @IBOutlet weak var tfDuration: UITextField!
    @IBOutlet weak var tfRating: UITextField!
    @IBOutlet weak var tvSummary: UITextView!

    override func viewDidLoad() {
        super.viewDidLoad()
        if let movie = movie {
            ivMovie.image = movie.image as? UIImage
            tfTitle.text = movie.title
            tfCategories.text = movie.categories
            tfDuration.text = movie.duration
            tfRating.text = "☆ \(movie.rating)/10"
            tvSummary.text = movie.summary
        }
    }

    override func touchesBegan(_ touches: Set<UITouch>, with
      event: UIEvent?) {
        view.endEditing(true)
    }

    @IBAction func addEditMovie(_ sender: UIButton) {
        //Se o movie for nulo é sinal que estamos criando um
        //filme e portanto vamos instanciar um novo Movie.
        //Como é uma classe Core Data, instanciamos passando
        //o context.
        if movie == nil {
            movie = Movie(context: context)
        }

        //Passamos para o movie os valores dos campos
        movie?.title = tfTitle.text
        movie?.categories = tfCategories.text
        movie?.duration = tfDuration.text
        movie?.rating = Double(tfRating.text!) ?? 0
        movie?.image = ivMovie.image
        movie?.summary = tvSummary.text

        do {
            //O método abaixo salva qualquer alteração
            //ocorrida no contexto, ou seja, altera o filme
            //ou grava um novo.
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            try context.save()

            //O comando abaixo solicita à Navigation
              //Controller que retorne à tela anterior.
            navigationController?.popViewController(animated:
               true)
        } catch {
            print(error.localizedDescription)
        }
    }
}
```

Código-fonte 12 – Classe AddEditViewController
Fonte: Elaborado pelo autor (2019)

Rode o aplicativo e cadastre algum filme. Você verá que aparece na listagem, mesmo que feche o aplicativo. Por enquanto não iremos gravar a imagem do filme, faremos isso em outro módulo.

Do nosso **CRUD**, nós já implementamos o *Read* (leitura de filmes do banco) e o *Create* (cadastro de filmes), faltando o *delete e o update*. O *delete* é bem simples, basta descomentarmos o método *func tableView(_ tableView: UITableView, commit editingStyle: UITableViewCell.EditingStyle, forRowAt indexPath: IndexPath*) que se encontra comentado na classe **MoviesTableViewController**. Descomente esse método e deixe-o como no código-fonte “Excluindo filmes”.

```
//Quando implementamos o método abaixo, podemos realizar um
Swipe Left na célula permitindo que ela seja excluída
override func tableView(_ tableView: UITableView, commit
editingStyle: UITableViewCell.EditingStyle, forRowAt
indexPath: IndexPath) {

    //Se o usuário apertou no botão delete
    if editingStyle == .delete {

        //Recuperamos o filme daquela célula
        let movie = fetchedResultsController.object(at:
           indexPath)

        //O context possui um método para exclusão de um
        //elemento
        context.delete(movie)
        do {
            try context.save()
        } catch {
            print(error.localizedDescription)
        }
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        }
}
```

Código-fonte 13 – Excluindo filmes
Fonte: Elaborado pelo autor (2019)

Para finalizar, vamos fazer a edição dos filmes. Na *View Controller* de visualização do filme (*ViewController),* vamos adicionar um botão que levará para a tela de edição.

Adicione um *Bar Button* Item no *Navigation Bar* dessa tela, do lado direito, e mude seu *System* Item para **Edit**. Agora, crie uma segue (**show**) para a tela **AddEditViewController.**

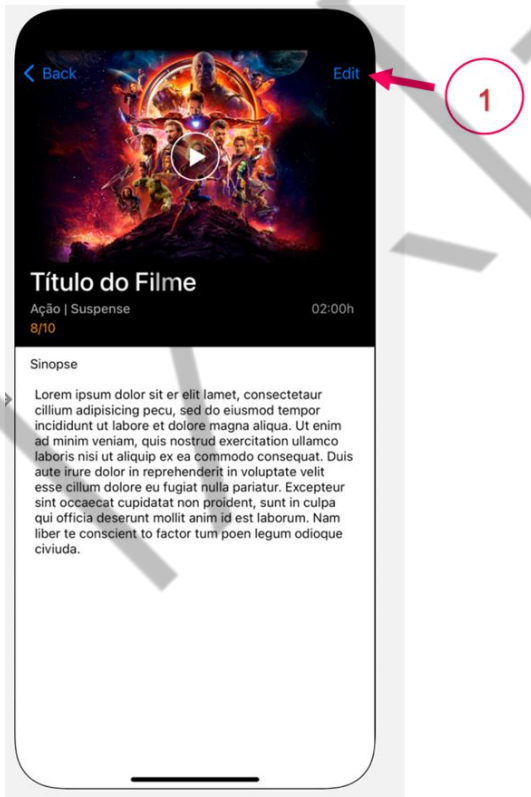

Figura 15 – Tela de Visualização de Filme com botão Edit
Fonte: Elaborado pelo autor (2022)

Vamos abrir a classe *ViewController* e implementar o método prepare, conforme o código-fonte “Enviando filme para a tela AddEditViewController”.

```
override func prepare(for segue: UIStoryboardSegue, sender:
Any?) {
    if let vc = segue.destination as? AddEditViewController {
        vc.movie = movie
    }
}
```

Código-fonte 14 – Enviando filme para a tela AddEditViewController
Fonte: Elaborado pelo autor (2019)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Agora, precisamos fazer algumas pequenas modificações. Na classe *AddEditViewController,* crie um *outlet* para o botão de cadastro (Cadastrar) e chame de **btAddEdit**. No método *viewDidLoad*, altere o texto desse botão, caso estejamos em modo edição. O início dessa classe ficará como mostra o código-fonte "AddEditViewController".

```
class AddEditViewController: UIViewController {

    //Variável que receberá o filme selecionado.
    var movie: Movie?

    @IBOutlet weak var ivMovie: UIImageView!
    @IBOutlet weak var tfTitle: UITextField!
    @IBOutlet weak var tfCategories: UITextField!
    @IBOutlet weak var tfDuration: UITextField!
    @IBOutlet weak var tfRating: UITextField!
    @IBOutlet weak var tvSummary: UITextView!
    @IBOutlet weak var btAddEdit: UIButton!

    override func viewDidLoad() {
        super.viewDidLoad()
        if let movie = movie {
            ivMovie.image = movie.image as? UIImage
            tfTitle.text = movie.title
            tfCategories.text = movie.categories
            tfDuration.text = movie.duration
            tfRating.text = "☆ \(movie.rating)/10"
            tvSummary.text = movie.summary

            //Usamos o método setTitle para alertar o texto
            //do botão
            //para o seu estado normal.
            btAddEdit.setTitle("Alterar", for: .normal)
        }
    }
```

Código-fonte 15 – AddEditViewController
Fonte: Elaborado pelo autor (2019)

Para concluir, volte para a classe *ViewController* e implemente o método *viewWillAppear.* Vamos tirar o código presente no *viewDidLoad* e colocar neste método, pois assim, sempre que a tela estiver prestes a aparecer, ela irá repopular os dados. Isso é útil quando alterarmos o filme e voltarmos para a tela anterior, pois assim

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

veremos as alterações. Ela ficará como apresentado no código-fonte “Nova versão de ViewController”.

```
import UIKit

class ViewController: UIViewController {

    var movie: Movie?

    @IBOutlet weak var ivMovie: UIImageView!
    @IBOutlet weak var lbTitle: UILabel!
    @IBOutlet weak var lbCategories: UILabel!
    @IBOutlet weak var lbDuration: UILabel!
    @IBOutlet weak var lbRating: UILabel!
    @IBOutlet weak var tvSummary: UITextView!

    override func viewDidLoad() {
        super.viewDidLoad()
    }

    override func viewWillAppear(_ animated: Bool) {
        super.viewWillAppear(animated)
        if let movie = movie {
            ivMovie.image = movie.image as? UIImage
            lbTitle.text = movie.title
            lbCategories.text = movie.categories
            lbDuration.text = movie.duration
            lbRating.text = "☆ \(movie.rating)/10"
            tvSummary.text = movie.summary
        }
    }

    override func prepare(for segue: UIStoryboardSegue,
            sender: Any?) {


        if let vc = segue.destination as?
                                AddEditViewController {
            vc.movie = movie
        }
    }
}
```

Código-fonte 16 – Nova versão de ViewController
Fonte: Elaborado pelo autor (2019)

Pronto, agora temos nosso **CRUD** 100% criado!

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.4 Persistindo dados com UserDefaults

Para concluir este módulo, vamos ver uma forma bem simples de persistirmos pequenas informações ou preferências, que é utilizando a classe **UserDefaults**. Essa classe é muito útil quando precisarmos, por exemplo, armazenar as preferências do usuário dentro do seu *App.*

A *UserDefaults* é uma tabela interna do seu *App* que trabalha com chaves e valores para armazenar as preferências do usuário. É como um grande dicionário de preferências. Na verdade, ele armazena os dados em formato de *Property List* (lista de propriedades).

Todas as informações armazenadas na *UserDefaults* são persistidas enquanto o aplicativo estiver instalado. Ao desinstalar, essas informações são também apagadas.

Pela sua simplicidade, não se deve armazenar informações sensíveis do usuário na *UserDefaults* (ex.: senhas, dados bancários etc.) pois não são armazenadas com nenhum tipo de segurança ou criptografia.

UserDefaults possui um método chamado *synchronize()*, que serve para persistir os dados imediatamente. O uso desse método não é obrigatório, pois a sincronia é feita automaticamente após um intervalo.

Para utilizarmos o *UserDefaults,* podemos acessá-lo por meio da propriedade de classe **standard**, que nos retorna um objeto compartilhado para nossa aplicação. Armazenamos nossas preferências por meio de métodos específicos para cada tipo, como por exemplo:

- func set(_ value: Any?, forKey defaultName: String)
- func set(_ value: Bool, forKey defaultName: String)
- func set(_ value: Double, forKey defaultName: String)
- func set(_ value: Float, forKey defaultName: String)
- func set(_ value: Int, forKey defaultName: String)
- func set(_ value: URL?, forKey defaultName: String)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Da mesma forma, recuperamos valores usando também métodos específicos para cada tipo:

- func object(forKey defaultName: String) -> Any?
- func string(forKey defaultName: String) -> String?
- func bool(forKey defaultName: String) -> Bool
- func double(forKey defaultName: String) -> Double
- func integer(forKey defaultName: String) -> Int
- func url(forKey defaultName: String) -> URL?

Vamos criar uma tela para configurarmos um ajuste em nosso App, que definirá se queremos ou não executar o trailer dos filmes automaticamente. Por enquanto, não iremos ver essa *feature* em ação, mas podemos testar a persistência dessa preferência no *device.*

Crie uma nova *View Controller* e adicione no *Storyboard.* Vamos, agora, fazer com que nosso *App* utitlize *Tab Bar Controller*. Para isso, selecione a *Navigation Controller* (clicando no Dragon Ball) e vá ao menu *Editor -> Embed In -> Tab Bar Controller.*

Agora, nossas telas estão dentro de uma *Tab Bar Controller*. Clique na *Tab Bar Controller* e vincule arrastando uma ligação da Tab Bar Controller até a cena recém-criada e escolha a opção view controllers.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

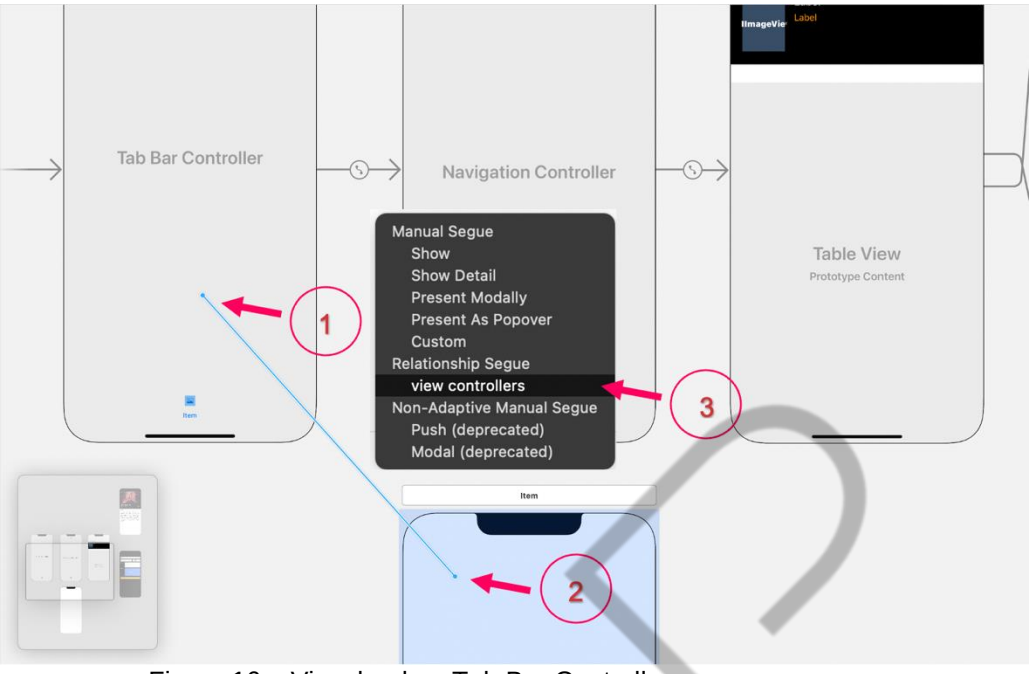

Figura 16 – Vinculando a Tab Bar Controller com a nova cena
Fonte: Elaborado pelo autor (2022)

Tente organizar seu *Storyboard* para que fique como mostra a Figura "Storyboard com Tab Bar Controller".

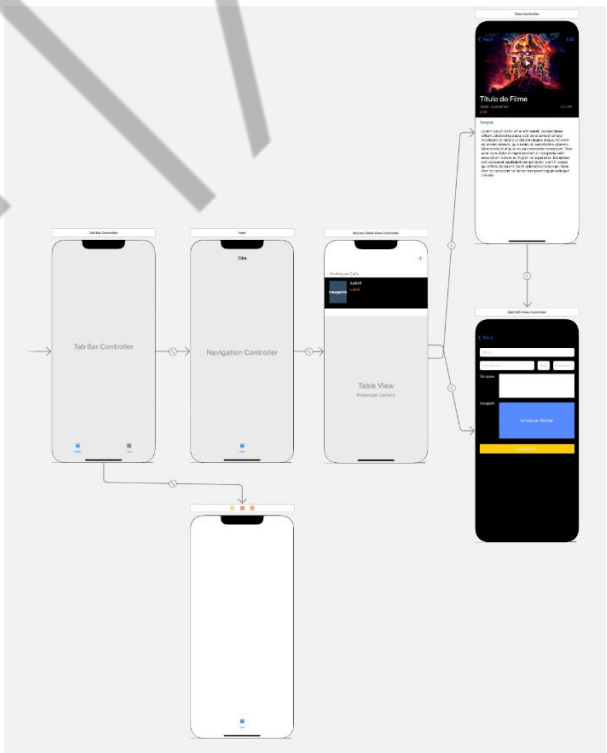

Figura 17 – Storyboard com Tab Bar Controller
Fonte: Elaborado pelo autor (2019)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Altere os *Tab Bar Itens* das duas cenas principais e faça com que a cena de filmes fique com o Title **Filmes** e com a Image **movies**, antes abra a pasta ícones disponível na área de Assets no portal e movimente pelo Finder as imagens map, movies e settings para a pasta Assets do seu Project Navigator.

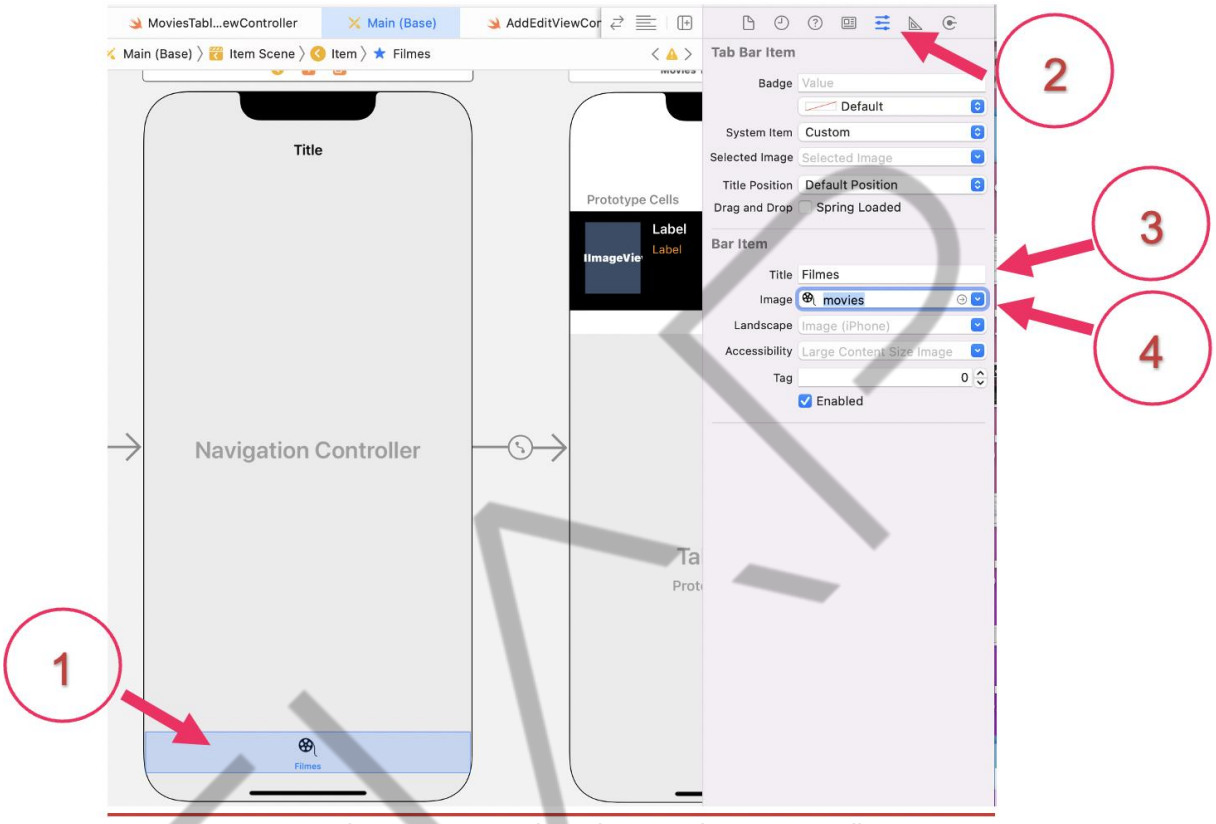

Figura 18 – Storyboard com Tab Bar Controller
Fonte: Elaborado pelo autor (2019)

Na nova cena, deixe o Title como **Ajustes** e a Image como **settings**.

Crie uma nova *Cocoa Touch Class* chamada **SettingsViewController**, herdando *UIViewController* e atribua esta classe a essa nova cena pelo Identity Inspector.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

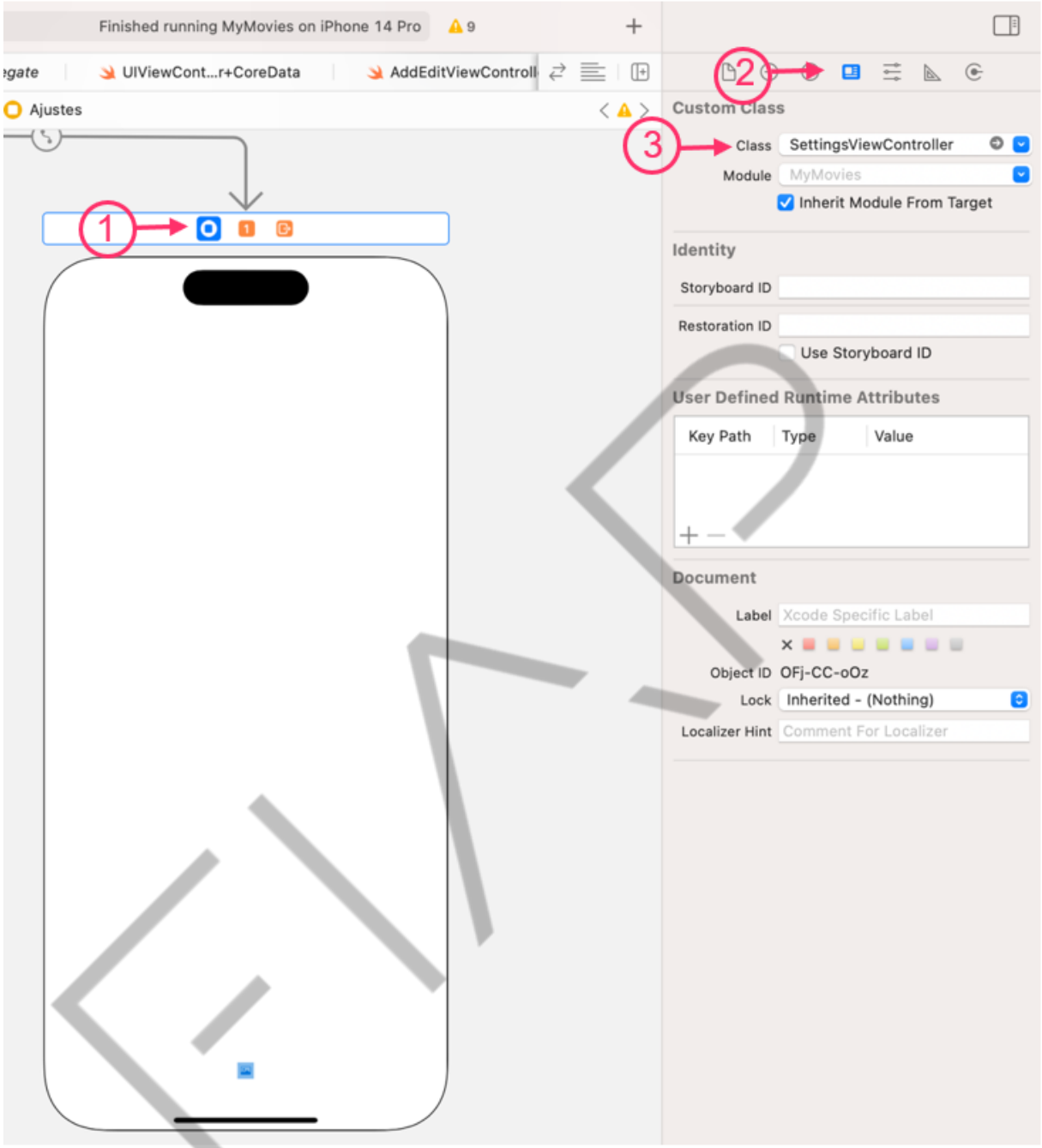

Figura 19 – Atribuindo a classe à nova cena via Identity Inspector
Fonte: Elaborado pelo autor (2023)

Nessa cena, adicione uma *Label* central com tamanho 23, escrito **Ajustes**. Adicione outra *label* com tamanho 15 e texto: “Rodar vídeos automaticamente?”. Ao lado dessa *label,* adicione o componente *Switch.* Esse componente é útil quando precisamos perguntar se o usuário quer habilitar ou não algo. Utilizamos a propriedade **isOn** para saber se ele está ligado ou não.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Crie um *outlet* para o *Switch* com o nome **swPlay** e uma *action* com Type UISwitch com o nome **changePlay**. O visual e código ficarão como na Figura “Tela de Ajustes”.

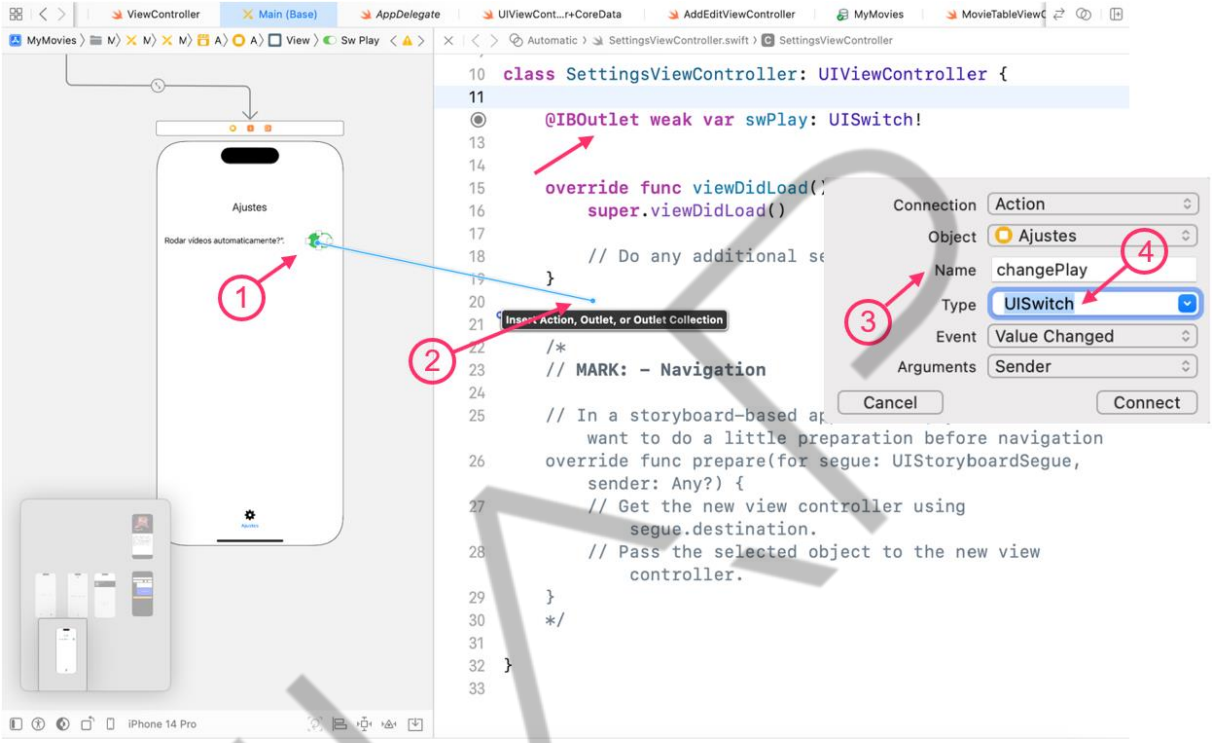

Figura 20 – Depois do Outlet definindo o Action com Name e Type
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

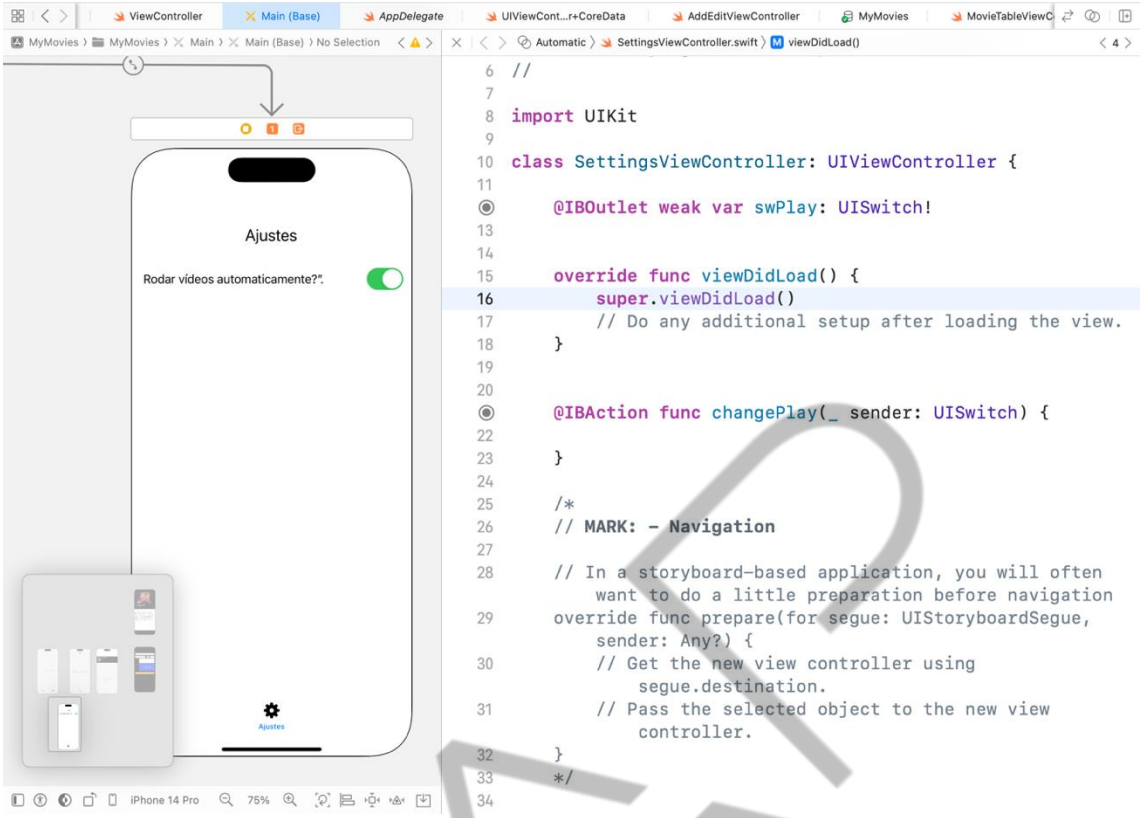

Figura 21 – Tela de ajustes
Fonte: Elaborado pelo autor (2023)

Agora, implemente o código-fonte “Nova versão de SettingsViewController”. Nós iremos recuperar a preferência do usuário (se quer ou não tocar os vídeos automaticamente) sempre que essa tela for criada e também iremos persistir o que o usuário escolher.

```
import UIKit

class SettingsViewController: UIViewController {

    @IBOutlet weak var swPlay: UISwitch!

    override func viewDidLoad() {
        super.viewDidLoad()

        //Aqui estamos pedindo ao UserDefaults nos fornecer
        //o booleano vinculado à chave play. Caso não exista
        //o valor será false.
        swPlay.isOn = UserDefaults.standard.bool(forKey:
            "play")
    }

    @IBAction func changePlay(_ sender: UISwitch) {

        //Sempre que o usuário tocar no switch, o seu estado
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        //(isOn é a propriedade que indica se o switch está
        //ligado ou não) será atribuído à chave play dentro
        //do UserDefaults.
        UserDefaults.standard.set(sender.isOn, forKey:
                "play")
    }

}
```

Código-fonte 17 – Nova versão de SettingsViewController
Fonte: Elaborado pelo autor (2019)

Com isso, concluímos a parte de persistência de dados de nosso aplicativo. Agora, podemos criar filmes, visualizá-los, editá-los e excluí-los, e também conseguimos persistir as preferências do usuário no App.

E em seu projeto, já pensou quais informações você gravará no aplicativo?

Se você ainda não pensou em uma tela para praticar o capítulo anterior e este capítulo, agora é a hora, aproveite este momento e comece seu app. Você precisa escolher apenas uma das plataformas (Android ou iOS) para começar e criar o quanto antes um projeto só seu.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

APPLE. **Framework UIKit** 2023. Disponível em: <https://developer.apple.com/documentation/uikit>. Acesso em: 26 jun. 2023.

APPLE. **Framework CoreData** 2023. Disponível em: <https://developer.apple.com/documentation/coredata>. Acesso em: 26 jun. 2023.

APPLE. **Framework UserDefaults** 2023. Disponível em: <https://developer.apple.com/documentation/foundation/userdefaults>. Acesso em: 26 jun. 2023.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com